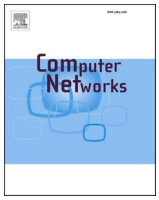
Computer Networks 290 (2026) 112774



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet



Fairness-aware and parallelized SFC placement with VNF sharing in MEC-enabled IoT networks

Qiang Zhu [a,*], MingZhu Li [a], Amin Rezaeipanah [b,*]

[a] School of Science, Engineering and Agriculture, Zhangjiajie College, Zhangjiajie, Hunan, 427000, China
[b] Department of Computer Engineering, University of Rahjuyan Danesh Borazjan, Bushehr, Iran

ARTICLE INFO

Keywords:
Service function chain
Network function virtualization
Network function parallelism
Fairness-aware resource allocation
VNF sharing
Deep reinforcement learning

ABSTRACT

Network Function Virtualization (NFV), as a fundamental enabler of Mobile Edge Computing (MEC), facilitates the decomposition of monolithic network services into modular components. These components are organized as Service Function Chains (SFCs) in MEC-enabled Internet of Things (IoT) networks, where Virtual Network Functions (VNFs) are arranged in a predefined order to ensure efficient service delivery. The key challenge lies in optimal SFC placement, where end-to-end latency often increases linearly with the length of SFCs. While Network Function Parallelism (NFP) mitigates latency by enabling parallel processing of packets across multiple VNFs, the combined effects of fairness in resource competition and the benefits of VNF sharing have been largely overlooked. To address this gap, we propose a Fairness-aware and Parallelized SFC Placement framework with VNF Sharing (FPPVS), formulated through deep reinforcement learning. FPPVS identifies interdependencies among VNFs to reconstruct traditional SFCs into partially parallelized configurations, while ensuring latency constraints are satisfied. Moreover, it ensures fairness in resource allocation while leveraging VNF instance reuse for future requests, taking into account latency effects caused by resource contention. Extensive simulations show that FPPVS achieves lower average latency and higher resource efficiency than existing methods.

1. Introduction

The rapid proliferation of Internet of Things (IoT) applications has placed unprecedented demands on network infrastructures, necessitating innovative paradigms to ensure efficient resource utilization, high reliability, and low latency [1]. In order to achieve strict service requirements, Mobile Edge Computing (MEC) has become a crucial enabler in this environment, bringing computing and storage resources closer to end users. However, the adoption of more adaptable and economical methods is encouraged by the substantial capital costs (CAPEX) and operational expenditures (OPEX) associated with establishing standard network topologies in MEC contexts [2–4]. By separating network services from specialized hardware and facilitating their deployment as modular, software-based components, Network Function Virtualization (NFV) has emerged as a crucial technology to achieve MEC. NFV makes it easier to deliver MEC services at a lower cost, increases scalability, and improves adaptability by virtualizing network capabilities into Virtual Network Functions (VNFs). Representative VNFs include Intrusion Detection Systems (IDS), Deep Packet Inspection (DPI), Load Balancers (LB), Firewalls (FW), and Network Address Translation (NAT), among others [5,6].

To guarantee orderly packet processing and smooth service delivery, VNFs are typically arranged into Service Function Chains (SFCs). In order for traffic flows to reach the intended Quality of Service (QoS), an SFC specifies a certain order of VNFs, which is frequently dictated by application requirements [7,8]. Deploying ordered VNFs onto physical nodes and creating the required connections while meeting latency and throughput requirements constitute the SFC placement problem. Despite the advantages of NFV, resource constraints and heterogeneity in MEC nodes, dynamic and massive IoT traffic, and the intrinsically sequential structure of SFCs that worsens end-to-end latency make optimal SFC placement in MEC-enabled IoT networks extremely difficult [2]. Additionally, equitable resource distribution is crucial, and VNF sharing complicates placement choices.

As the number of VNFs in an SFC increases, end-to-end latency grows due to sequential processing at each VNF [1]. Processing latencies can be greatly decreased by sharing existing VNF instances during SFC assembly, which reuses resources rather than creating new ones. By identifying shared VNFs and rerouting traffic to instances that are already operational, this method lowers the overhead associated with deploying

* Corresponding authors.
E-mail addresses: happy_zhQ2025@163.com (Q. Zhu), limingzhu_2025@163.com (M. Li), amin.rezaeipanah@gmail.com (A. Rezaeipanah).

https://doi.org/10.1016/j.comnet.2026.112774
Received 27 May 2025; Received in revised form 18 July 2026; Accepted 8 September 2026
Available online 11 September 2026
1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

new instances, conserves computational resources, and speeds up operations. SFC placement is made more difficult by the intricate limits introduced by VNF sharing. It could impair user Quality of Experience (QoE) and stop SFC requests from being deployed along the shortest path [4,9]. Longer pathways use more link resources, which reduces MEC's ability to handle incoming requests. Additionally, changing instance distributions over time and MEC resource restrictions must be taken into account for successful VNF sharing, which is further complicated by network dynamics and intricate request patterns [10]. Resource management techniques and fairness principles are crucial for maintaining QoS in order to avoid congestion and excessive latencies brought on by competing flows when sharing VNFs [11]. As a result, VNF sharing might not always provide the best answer for the current SFC request.

Adoption of NFV can be incorporated with little disturbance and does not require a total redesign of network infrastructure. However, the abstraction layer that is added between VNFs and hardware results in extra overhead, which lowers performance and increases delay. End-to-end latency increases linearly with the number of VNFs in an SFC [1–3], and VNF processing generally stays slower than hardware-based implementations even with enhancements like Data Plane Development Kit (DPDK) and Direct Data I/O (DDIO) [12]. For latency-sensitive applications like remote healthcare or autonomous driving, this linear increase is especially problematic. To reduce end-to-end latency, a number of research have concentrated on traffic scheduling, VNF acceleration, resource utilization, and VNF instantiation. Although these works offer useful first insights, they frequently assume linear acceleration and sequential execution of VNFs within a single SFC, which simplifies coordination but ignores potential parallelism among VNFs like Caching and NAT, which can function concurrently. Nearly half of VNF pairings can be processed concurrently without a precise sequence, according to empirical data [9]. Parallelized VNF placement has only been attempted by a few numbers of conventional techniques, with only modest performance gains under increasing SFC demands. Furthermore, in order to maximize p-SFC coordination, current methods typically rely on prior information and are unable to dynamically and intelligently capture hidden dependencies between VNFs or changeable network states.

Recent studies have investigated Network Function Parallelism (NFP), which enables certain VNFs to handle traffic concurrently rather than strictly sequentially, in order to overcome intrinsic sequential delay [13]. As seen in Fig. 1, NFP can be used to lower latency for some VNFs that support parallel execution, like caching and NAT. In order to convert a serial/sequential SFC (s-SFC) into a parallelized SFC (p-SFC), NFP finds parallelizable VNF instances where packets can be processed concurrently, divides traffic among them, and merges outputs [14]. Shorter routing paths and lower end-to-end latency are made possible by deploying VNFs based on NFP. However, as NFP may result in increased memory usage, extra packet transmissions, traffic loads, and packet loss, the conversion of an s-SFC to a p-SFC must maintain overall SFC integrity. Even though NFP has the potential to reduce latency, current solutions frequently ignore two important factors: VNF sharing across numerous requests and resource allocation fairness. While ignoring VNF reuse during parallelization loses potential for increased resource efficiency, ignoring fairness can result in resource monopolization [13]. Because numerous VNFs vying for shared server and link resources can raise processing latencies, lower performance, and possibly break Service Level Agreements (SLAs), reducing end-to-end latency alone is insufficient for p-SFC implementations [14]. In order to handle more SFC requests and balance network performance, scalability, and stability, a fairness-aware solution guarantees equal resource allocation while preserving latency limitations.

The SFC placement is generally an NP-hard problem, and Deep Reinforcement Learning (DRL) [15–17] has gained popularity due to its adaptability to dynamic environments. Nevertheless, computational cost and the extra resources required for packet replication and merging present difficulties for DRL solutions that include VNF sharing with NFP [18]. Furthermore, it is impossible to overlook erratic resource variations and traffic needs. Federated DRL (FDRL) [19,20], which integrates DRL and federated learning, has demonstrated encouraging results in a number of applications. FDRL is a promising method for SFC placement and coordination because neural networks in DRL and federated learning can learn correlations among VNFs, network states, and performance rewards. However, model-based VNF placement learning encounters efficiency issues and increased resource usage as SFC counts and network state complexity rise. Current approaches, such as federated learning, are time-consuming and unable to dynamically take VNF independence into account for parallelized placement.

These challenges motivate the need for a unified framework that combines fairness, SFC parallelization, and VNF sharing. To address this issue, we propose the Fairness-aware and Parallelized SFC Placement with VNF Sharing (FPPVS) framework. FPPVS uses Federated Deep Reinforcement Learning (FDRL) to make adaptive placement decisions under dynamic network conditions. It converts sequential SFCs into partially parallel SFCs whenever dependencies allow, reducing end-to-end latency through concurrent VNF execution. At the same time, it reuses existing VNF instances whenever possible to reduce deployment cost and improve resource utilization. A fairness-aware resource allocation strategy prevents a small number of SFCs from monopolizing network resources. FPPVS also decides which VNFs should run in parallel, which instances can be shared, and where new instances should be deployed while satisfying latency and resource constraints. Finally, the

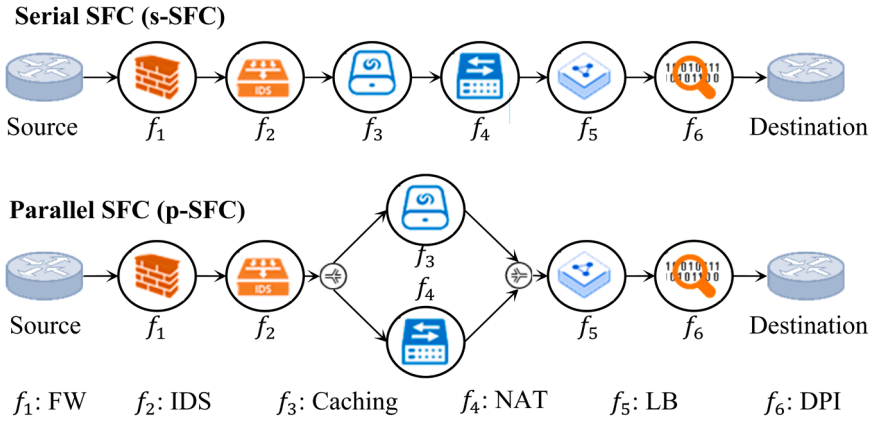


Fig. 1. s-SFC versus p-SFC. The upper figure illustrates an s-SFC composed of six ordered VNFs. The lower figure presents its corresponding p-SFC, derived from the sequential representation. The p-SFC consists of a main chain, *Source*→f_1→f_2→f_5→f_6→*Destination*, along with two branch chains: f_2→f_3→f_5 and f_2→f_4→f_5. In each branch chain, the first and last VNFs serve as the ingress and egress points, respectively.

FDRL agent continuously learns from previous placement decisions to improve throughput, scalability, and overall network performance.

The key contributions of this study are given below:

- FPPVS is a novel SFC placement framework that simultaneously integrates fairness, NFP, and VNF sharing to optimize resource utilization in MEC-enabled IoT networks. It transforms s-SFCs into partially parallelized structures for concurrent packet processing, strategically reuses existing VNFs to minimize overhead and latency, and employs fairness-aware resource allocation across networks to ensure efficient utilization while maintaining QoS and equitable service delivery.
- A FDRL framework is proposed to train a global model across orchestrators and the cloud using latency- and reward-based aggregation. SFCs are decomposed according to network resources and parallelizable VNF pairs for fast convergence, while VNF parallelism rules ensure correct processing and reduced end-to-end latency. Also, to speed up FDRL convergence, a reward-driven adaptive ϵ-greedy strategy is employed, enabling faster exploration of all possible actions at each state.
- We show that FPPVS dramatically lowers end-to-end latency, improves resource efficiency, and beats cutting-edge SFC placement methods through extensive simulations across several MEC-enabled IoT situations.

The remainder of this paper is organized as follows. Section 2 reviews related work on SFC placement. Section 3 presents the system model and problem formulation. Section 4 describes the proposed FPPVS model. Section 5 reports and discusses the performance evaluation results. Finally, Section 6 concludes the paper and outlines future research directions.

2. Related work

A wide spectrum of approaches has been investigated in the literature for the SFC placement problem, including evolutionary algorithms [21,22], hybrid models [23], clustering-based techniques [24–26], neural networks [27,28], and more recently, machine learning-based solutions [29]. This study classifies existing solutions into non-DRL and DRL-based approaches and summarizes the relevant literature. Table 1 summarizes related works.

2.1. Non-DRL approaches

Traditionally, the SFC placement problem has been formulated as a mathematical optimization task solvable by exact methods. While exact solvers can provide optimal solutions, their high computational cost limits their applicability in large-scale or dynamic networks. Heuristic and meta-heuristic methods, which provide near-optimal solutions with much lower complexity and quicker execution times, have been frequently used to overcome problem. Despite their effectiveness, these techniques frequently sacrifice optimality, and in dynamic, diverse contexts where scalability and flexibility are crucial, their performance may deteriorate [44,45].

Static SFC deployment and resource allocation were the main topics of early research. In order to guarantee dependability, Liu et al. [14] suggested a latency-minimization strategy in heterogeneous edge environments by simultaneously optimizing SFC deployment and resource management. In order to enhance resource utilization and minimize the number of servers needed, Li et al. [46] created a correlation-based greedy method for mapping SFC requests to actual machines. In order to optimize p-SFC deployment, Zhang et al. [47] presented a dynamic programming methodology inspired by Viterbi, which resulted in reduced latency and effective resource usage when compared to conventional greedy methods. In order to reduce VNF deployment costs, energy consumption, traffic forwarding costs, and SLA violations, Bari

Table 1
Summary of related works.

Reference	Model	Methodology	Strengths	Weaknesses
[30]	READ	GSP for reliability-aware, latency-constrained routing	Maximizes service reliability, minimizes end-to-end latency	Heuristic may not scale well under highly dynamic traffic
[31]	DSMO	LP and evolutionary approaches	Jointly optimizes VNF deployment and resource allocation, minimizes network latency	LP be less adaptable to dynamic network conditions
[32]	NSE	ILP-based multi-provider NSE with SFC graph partitioning	Significant savings in service cost and resource usage	ILP is computationally intensive and less scalable
[33]	-	Game theory-inspired graph partitioning	Ensures Nash equilibrium, satisfies server affinity, collocation, and latency constraints	Complexity increases with large-scale networks
[34]	EPSFC	Heuristic for energy-efficient and latency-aware deployment	Co-locates parallel VNFs, minimizes server/link usage, reduces energy consumption, supports more PSFC requests	Heuristic not guarantee globally optimal solutions
[35]	ParaSFC	Heuristic SFC using Viterbi dynamic programming	Reduces latency, near-optimal deployment, identifies bottlenecks effectively	Heuristic approach; Not scale for extremely large networks
[36]	rzMODRL	Multi-objective DRL with encoder-decoder LSTM	Handles high-dimensional inputs, improves generalization, avoids retraining in lower-dimensional scenarios	Training complexity; Require substantial computation
[37]	-	Deep Q-learning with fBm traffic modeling and GPR	Dynamically adapts to non-stationary traffic, satisfies latency requirements	Complex traffic modeling; potential computational overhead
[38]	APSD	Actor-critic DRL framework	Optimizes service latency, deployment cost, and acceptance rate	Require extensive training and fine-tuning
[39]	DRL-SFCP	A3C-based DRL with GCN + Seq2Seq	Captures network topology and ordered SFC info; maximizes long-term revenue	High computational complexity; sequential placement may propagate errors
[40]	RVNF	A3C-based DRL	Balances long-term cumulative reward with service quality	Heavily relies on accurate reward modeling

(continued on next page)

Table 1 (*continued*)

Reference	Model	Methodology	Strengths	Weaknesses
[41]	PVFP	FDRL-based NFP	Enables partial VNF parallelism, balances multi-domain resources, federated global model training	Overhead from federated aggregation; complexity in dependency management
[42]	DPC	Distributed SFC with A3C	Simultaneous activation of non-conflicting VNFs, improved resource utilization and capacity	Managing dependencies across distributed servers can be complex
[43]	PVRDU	DRL-based NFP under Resource Demand Uncertainty	Handles high latency and uncertain resource demands, transforms SFCs to parallel configurations, low-latency deployment	Complexity in DRL training and Markov approximation; computational overhead

et al. [48] structured the problem as an Integer Linear Programming (ILP) and suggested a dynamic programming-based heuristic. Similarly, Li et al. [49] presented a latency-aware VNF migration method using a hybrid genetic evolution algorithm, significantly reducing average network latency.

A number of studies expanded SFC placement to take multi-domain settings, latency restrictions, and dependability into account. For NFV-enabled datacenter networks, Qu et al. [30] introduced a reliability-aware and latency-constrained (READ) routing system that uses the heuristic Greedy-k-shortest paths (GSP) algorithm to minimize end-to-end latency and maximize service reliability. In order to improve provider profit through VNF migration, reinstantiation, and rescheduling while taking heterogeneous node resources and migration costs into account, Li et al. [50] investigated dynamic SFC mapping and scheduling in Internet of Vehicles (IoV) services and introduced Tabu Search-based methods. In their investigation of the relationship between resource consumption and SFC latency, Liu et al. [31] proposed a Discrete Spider Monkey Optimization (DSMO) approach to reduce overall network latency and formulated a joint VNF deployment and resource allocation problem as a linear programming model.

Specialized heuristics and orchestration techniques were investigated in other investigations. While Fang et al. [51] enhanced SFC embedding through joint state and VNF orchestration, Asgarian et al. [52] addressed linearization of non-linear constraints to reach near-optimal solutions. Liu et al. [53] presented a Lagrangian relaxation heuristic for dynamic orchestration, improving latency performance and service acceptance, and Thanh et al. [54] suggested an energy-sensitive deployment technique for edge-cloud settings for IoT-specific situations. In their investigation of multi-provider Network Service Embedding (NSE), Dietrich et al. [32] showed notable reductions in service cost and resource consumption by breaking down the problem into SFC graph partitioning and mapping using ILPUsing a graph partitioning technique influenced by game theory, Leivadeas et al. [33] demonstrated the existence of a Nash equilibrium to meet server affinity, collocation, and latency restrictions for SFCs. EPSFC, a heuristic for the energy-efficient and latency-conscious deployment of p-SFCs in NFV networks, is proposed by Chintapalli et al. [34]. It optimizes server and link utilization, ensures end-to-end latency while lowering energy consumption and allowing more PSFC queries, co-locates parallel VNFs, and dynamically distributes resources based on a resource–latency model. In order to achieve near-optimal deployment with lower latency, Luo et al. [35] present ParaSFC, a heuristic method that turns s-SFCs into parallelized configurations by utilizing VNF dependencies and the Viterbi dynamic programming algorithm to find bottlenecks and modify processing sequences.

A number of complimentary approaches to enhancing intelligent network service management have been identified by recent research. Through autonomous decision-making and fault prediction, adaptive predictive frameworks have been proposed to improve the reliability of large-scale distributed systems [55]. Memory-efficient packet processing techniques have been implemented at the network infrastructure level to enhance scalability and lower processing overhead in high-speed multi-tenant networks [56]. Recent developments in mobile and edge environments have concentrated on DRL-based resource optimization under dynamic wireless situations [57], secure task offloading in vehicular edge computing [58], collaborative edge intelligence for big AI models [59], and mobility-aware connectivity management [60].

Although these optimization- and heuristic-based approaches provide useful answers in specific situations, they are still not very flexible when applied to diverse, large-scale, and dynamic networks. While heuristic procedures frequently compromise optimality, especially as networks grow and SFC diversity rises, exact solutions are computationally impractical in complicated situations. These restrictions have spurred research into more flexible and learning-based approaches that can balance efficiency, scalability, and service quality while reacting dynamically to network changes.

2.2. DRL approaches

DRL has emerged as a potent technique for adaptive and intelligent resource allocation in the extensive research on the effective deployment of SFCs in networks allowed by MEC and NFV [44,45]. Compared to conventional static or rule-based approaches, DRL can efficiently capture dynamic network conditions and offer more scalable and effective placement techniques by continuously engaging with the network environment and learning from previous experiences.

DRL has been used in a number of studies to address different issues of SFC placement. With an emphasis on multi-agent learning to increase service provider profitability, Zahedi et al. [61] use DRL for dynamic SFC placement and resource allocation in fog and edge computing. A deep learning-based system for proactive scaling of VNF resources by forecasting processor, memory, and bandwidth demands is proposed by Taha et al. [62]. A DRL method based on an enhanced deep deterministic policy gradient is presented by Liu and Zhang [63], which achieves faster convergence and a latency reduction of more than 28%. In order to accommodate high-dimensional inputs, improve generalization, and prevent retraining in lower-dimensional settings, Zhou et al. [36] present a similar zone Multi-Objective DRL (rzMODRL) architecture that uses an encoder-decoder Long Short-Term Memory (LSTM) model. In order to meet latency restrictions, Qu et al. [37] integrate traffic modeling with deep Q-learning to handle dynamic VNF scaling and migration under non-stationary traffic. Their method uses a fractional Brownian motion (fBm) traffic model, which allows for precise adaptation to traffic fluctuations by learning traffic parameters for each stationary segment of a non-stationary trace utilizing change point detection in conjunction with Gaussian Process Regression (GPR).

Other noteworthy DRL-based solutions concentrate on multi-objective and QoS-aware optimization. An online SFC deployment strategy that balances load, controls resource consumption, and guarantees QoS for Internet of Things applications is proposed by Xu et al. [64] for cloud-edge collaborative architectures. The Autoscaling and Priority-aware SFC Deployment (APSD) framework, introduced by Yu et al. [38], uses an actor-critic DRL model to maximize acceptance rate, deployment cost, and service latency. Similar to this, Wang et al. [39]

present DRL-SFCP, which maximizes long-term average revenue by utilizing an Asynchronous Advantage Actor-Critic (A3C) framework with a Graph Convolutional Network (GCN) and a Seq2Seq model with gated recurrent units (GRUs) to capture network topology and ordered SFC information, respectively.

Abedi et al. [65] proposed a two-stage auction framework for resource orchestration in multi-tenant 6G network slicing. Their method combines resource sharing with distributed reinforcement learning to improve utility and reach a stable resource allocation. Pourghasemian et al. [66] studied VNF placement and resource allocation in UAV-assisted NFV networks. They used a hierarchical DRL approach to jointly optimize UAV trajectories, VNF scheduling, and migration, reducing both energy consumption and service latency. Akbari et al. [67] addressed VNF scheduling in industrial IoT networks. Their DRL-based framework minimizes the Age of Information (AoI), and the multi-agent version achieves better network cost and information freshness than conventional and greedy methods. Zhang et al. [40] propose RVNF, a DRL-based approach using A3C to optimize dynamic VNF placement, enhancing VNF reuse and balancing resource costs, latency, and throughput while penalizing failed deployments to ensure efficient SFC execution.

In parallel, NFP has gained attention as a method to enhance SFC performance by enabling concurrent processing of VNFs. Huang et al. [41] propose the Parallel VNF Placement (PVFP) framework, which uses FDRL with three parallelism identification rules, partitions SFCs across multi-domain resources for balanced deployment, and trains a global model via latency- and reward-based federated aggregation. Building on similar principles, Guo and Rezaeipanah [42] develop the Distributed Parallel Chaining (DPC) algorithm, which simultaneously activates non-conflicting VNFs and employs the A3C framework to maximize cumulative rewards and improve resource utilization across distributed servers. The problem of high latency and variable resource demand in SFC deployment within MEC networks is addressed by Zhang et al. [43]. The Parallelizing VNFs under Resource Demand Uncertainty (PVRDU) framework is suggested by the authors. PVRDU assembles sub-SFCs using a DRL-based method and uses dependencies between VNFs to convert conventional s-SFCs into parallelized configurations. To further provide more dependable and low-latency SFC deployment, a Markov-based approximation approach is utilized to efficiently control resource demand uncertainty.

Despite these developments, there are still a number of difficulties with SFC placement based on DRL and NFP. Current DRL techniques frequently rely on Seq2Seq models that cannot effectively manage sequential dependencies, underuse graph-based neighborhood information to represent network states, and suffer from error propagation because of VNF-by-VNF insertion. Although promising, NFP-based methods have computational overhead and complicated VNF dependency management. The creation of frameworks that incorporate fairness, parallelism, and VNF sharing for more effective and scalable SFC placement is motivated by these constraints.

3. System model

The SFC is modeled as a Directed Acyclic Graph (DAG) to facilitate efficient traffic routing across VNFs and allow flexible deployment of new services. For clarity, the notations are summarized in Table 2, followed by the detailed system model description.

Network Model: The underlying MEC substrate network is represented as an undirected graph $G_s = (V_s, E_s)$, where V_s denotes the set of data centers (DCs) and E_s represents the set of physical links between them. Each $v_i \in V_s$ corresponds to a DC equipped with computational resources C_i, while each $e_{i,j} \in E_s$ denotes a physical link between v_i and v_j, characterized by communication latency $L_{i,j}$ and bandwidth capacity $B_{i,j}$. The latency between two DCs is modeled as the total latency along the shortest path connecting them. Furthermore, let $\mathbb{V} = \{\mathcal{v}_1, \mathcal{v}_2, \ldots, \mathcal{v}_{|\mathbb{V}|}\}$ denote the set of available VNF types within G_s, where $|.|$ indicates the cardinality of a set (e.g., $|\mathbb{V}|$ is the number of VNF types). In this study, a dependency matrix CD is introduced to ensure resource isolation and to prevent conflicts when two VNFs are executed concurrently. Each element $cd_{j,k} \in CD$ is defined as follows:

$$cd_{j,k} = \begin{cases} 1 & \textbf{if } \mathcal{v}_j \textbf{ is dependent on } \mathcal{v}_k \\ -1 & \textbf{if } \mathcal{v}_k \textbf{ is dependent on } \mathcal{v}_j \\ 0 & \textbf{otherwise} \end{cases} \tag{1}$$

where $\mathcal{v}_j$ represents the j-th VNF. It is assumed that the dependency relationships among all VNFs in the network are known in advance. For a more detailed discussion of VNF dependency rules, the reader is referred to [35].

SFC Requests: The set of SFC requests is denoted as $U = \{u_1, u_2, \ldots, u_{|U|}\}$, where $|U|$ represents the number of requests within the time interval Δt. Each SFC request is assumed to arrive at the network in a time slot t. Here, a sequence of discrete time slots $\{0, 1, \ldots, t, \ldots, T-1\}$ is considered, where in each slot the packets originating from the source node are sequentially processed by multiple VNF instances and then delivered to the destination node. Since the focus is placed on a single request, the time index t is omitted from the model for simplicity. Each request $u_i \in U$ is defined as a 7-tuple $\langle G_v, t_{arr}, t_{ttl}, r_\Delta, \ell, \mathscr{P}, v_{src}, v_{dst} \rangle$. Here, $G_v = (V_v, E_v)$ represents the SFC request as a directed graph, where V_v denotes the set of VNFs and E_v represents the set of virtual links between them. Each $\mathcal{v}_j \in V_v$ is associated with a computational demand D_j and a processing time L_j, while each $e_{i,j} \in E_v$ requires a bandwidth demand B_e. Specifically, the ordered set of VNFs in G_v is given by $V_v = \{\mathcal{v}_1, \mathcal{v}_2, \ldots, \mathcal{v}_{|V_v|}\}$, where $|V_v|$ denotes the number of VNFs in the request. In addition, t_{arr} indicates the arrival time of the request, t_{ttl} denotes its lifetime, r_Δ is the requested data rate, ℓ specifies the average throughput to satisfy QoS and SLA requirements, and $\mathscr{P}$ represents the payment associated with the SFC request. The parameters v_{src} and v_{dst} refer to the source and destination nodes of the service path, respectively. For modeling purposes, $\mathcal{v}_{src}$ and $\mathcal{v}_{dst}$ are treated as two auxiliary VNFs with zero computational demand and processing time, connected to the head and tail of the SFC. Consequently, two additional edges, $e_{\mathcal{v}_{src}, \mathcal{v}_1}$ and $e_{\mathcal{v}_{|V_v|}, \mathcal{v}_{dst}}$, are incorporated into G_v. The objective is to map each $\mathcal{v}_j \in V_v$ onto a physical node $v_i \in V_s$ such that the latency requirements of the SFC are satisfied, while respecting the resource constraints of the underlying network.

SFC Deployment: The deployment of VNFs involves determining how VNFs are distributed across different DCs. The internal assembly of VNFs within a DC is carried out by the corresponding Virtual Infrastructure Manager (VIM), which maps VNFs onto computing nodes [38]. In the SFC placement problem, the focus lies on inter-DC deployment

Table 2
Notations.

Notations	Description
V_s, v_i	Set of DCs (nodes or servers), i-th DC
E_s, E_v	Set of physical links between DCs, set of virtual links between VNFs
$\mathbb{V}$, V_v, $\mathcal{v}_j$	Set of available VNF types, set of VNFs in request, j-th VNF
U, u_k	Set of SFC requests, k-th SFC request
$L_{i,j}$ and $B_{i,j}$, C_i	Communication latency and bandwidth capacity between v_i and v_j, computational resources in v_i
D_j, L_j, B_e	Resources demanded for $\mathcal{v}_j$, processing time for $\mathcal{v}_j$, bandwidth demanded for each pair of VNFs
$cd_{j,k}$	Variable to determine the dependence between $\mathcal{v}_j$ and $\mathcal{v}_k$
$x_{i,j}$	Variable determining the placement of $\mathcal{v}_j$ on v_i
v_{src}, v_{dst}	Source and destination nodes of the service path
$\mathcal{v}_{src}$, $\mathcal{v}_{dst}$	Auxiliary VNFs connected to the head and tail of the SFC
y_u, ℓ_u	Successful deployment of the SFC u, average throughput of SFC u
$r_i^{(u)}(t)$	Input data rate v_i for post-processing request u at time t
$\varphi_j(u, \mathcal{v}_j, v, \mathcal{v}_{j+1})$	Path formation state between $\mathcal{v}_j$ and $\mathcal{v}_{j+1}$ that are deployed on u and v
$\mathbb{M}$, $\mathbb{B}$	Main chain in p-SFC, set of branch chains in p-SFC

decisions, while the intra-DC assembly process is not explicitly considered. Moreover, the mapping of virtual links in E_v is jointly decided during the VNF deployment process. Specifically, the shortest path between the allocated DCs is selected to map virtual links onto the substrate links E_s. If $\epsilon_j \in V_v$ is assigned to $v_i \in V_s$, then $x_{i,j} = 1$; otherwise, $x_{i,j} = 0$. The auxiliary VNFs ϵ_{src} and ϵ_{dst} are fixed on nodes v_{src} and v_{dst}, respectively. Hence, $x_{v_{src},\epsilon_{src}} = 1$ and $x_{v_{dst},\epsilon_{dst}} = 1$. In addition, the binary variable $tf^{p,q}_{j,j+1} \in \{0,1\}$ indicates whether the traffic flow between ϵ_j and ϵ_{j+1} traverses link $e_{p,q}$. When two consecutive VNFs ϵ_j and ϵ_{j+1} are deployed in the same DC, no bandwidth is consumed. The placement status of two consecutive VNFs is formulated as:

$$\hat{x}_{i,j} = x_{i,j} - x_{i,j+1} \tag{2}$$

where $\hat{x}_{i,j} = 1$ denotes that both ϵ_j and ϵ_{j+1} are deployed on the same DC v_i; otherwise, $\hat{x}_{i,j} = 0$. If all VNFs in request u are successfully deployed, then $y_u = 1$; otherwise, $y_u = 0$. More specifically, the decision variable y_u is defined as:

$$y_u = \begin{cases} 1 & \text{if } \left|V_v^{(u)}\right| = \sum_{v_i \in V_s} \sum_{\epsilon_j^{(u)} \in V_v^{(u)}} x_{i,j}^{(u)} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

where the superscript u denotes the u-th SFC request.

When two consecutive VNFs ϵ_j and ϵ_{j+1} are deployed, a routing path between them must be established. The construction of the path $\epsilon_j \rightarrow \epsilon_{j+1}$ includes both the outgoing flow from ϵ_j and the incoming flow to ϵ_{j+1}. Let $\varphi_j(u, \epsilon_j, v, \epsilon_{j+1})$ denote the path formation state between two consecutive VNFs, subject to the following constraints for both auxiliary and real VNFs [13]:

$$\sum_{u \in \Gamma(v_{src}),\ \chi \in V_v / \epsilon_{src}} \varphi_1(v_{src}, \epsilon_{src}, u, \chi) = 1 \tag{4}$$

$$\sum_{u \in \Gamma(v_{dst}),\ \chi \in V_v / \epsilon_{dst}} \varphi_{|V_v|}(u, \chi, v_{dst}, \epsilon_{dst}) = 1 \tag{5}$$

$$\sum_{u \in V_v,\ v \in \Gamma(u),\ \chi \in V_v / \epsilon_{src}} \varphi_j(u, \epsilon_j, v, \chi) = 1,\ \forall \epsilon_j \in V_v \Big/ \epsilon_{|V_v|} \tag{6}$$

$$\sum_{u \in V_v,\ v \in \Gamma(u),\ \chi \in V_v / \epsilon_{dst}} \varphi_j(u, \chi, v, \epsilon_{j+1}) = 1,\ \forall \epsilon_j \in V_v \Big/ \epsilon_{|V_v|} \tag{7}$$

where $\Gamma(v)$ denotes the set of neighbors of node v, and V_v / ϵ denotes the set of VNFs excluding ϵ.

Finally, the mapping of $x_{i,j}$ considering other VNFs is defined as:

$$x_{i,j} = \sum_{u \in \Gamma(v_i),\ \chi \in \epsilon_j : \forall j = 2 \ldots |V_v| - 1} \varphi_j(u, \epsilon_j, v_i, \chi) \tag{8}$$

Constraints: Each VNF from every SFC must be exclusively assigned to a single DC:

$$\sum_{v_i \in V_s} x_{i,\epsilon}(t) = 1,\ \forall \epsilon \in \mathbb{V},\ t \in \Delta t \tag{9}$$

Service completeness must be ensured during SFC assembly. This constraint guarantees that all VNFs of a given SFC are successfully deployed:

$$\sum_{v_i \in V_s} x_{i,j}^{(u)}(t) = 1,\ \forall \epsilon_j \in V_v,\ u \in U,\ t \in \Delta t \tag{10}$$

The computational workload of each DC and the bandwidth of each physical link at any time slot must not exceed their respective capacities. These constraints are formulated as:

$$\sum_{u \in U} \sum_{\epsilon_j \in V_v} x_{i,j}^{(u)} . D_j \le C_i,\ \forall v_i \in V_s \tag{11}$$

$$\sum_{u \in U} \sum_{\epsilon_{i,j} \in E_v} \sum_{\epsilon_j \in V_v} tf^{p,q}_{j,j+1} . B_e \le B_{p,q},\ \forall e_{p,q} \in E_s \tag{12}$$

End-to-end latency minimization during SFC assembly is essential to satisfy QoS requirements and ensure SLA compliance. Thus, DCs and physical links must provide sufficient resources for ongoing SFC requests without exceeding latency thresholds:

$$L_P(e_{i,j}) + L_V(e_{i,j}) + L_C(e_{i,j}) \le \ell,\ \forall e_{i,j} \in E_s \tag{13}$$

where L_P, L_V, and L_C represent propagation latency, virtualization latency, and processing latency, respectively.

Objective: Maximizing the sustainable acceptance ratio of SFC requests is the main objective, which can only be accomplished if all latency and resource criteria are met:

$$O_{acc} = \frac{1}{|U|} \sum_{u \in U} y_u \tag{14}$$

Reducing the overall cost of SFC deployment while maintaining the predetermined budget $\mathscr{P}$ is another important objective:

$$O_{cost} = \sum_{v_i \in V_s} (ut_i . PR_i + C_i . PD_i),\ \forall O_{cost} \le \mathscr{P} \tag{15}$$

where PR_i denotes the resource cost of DC v_i for VNF allocation, ut_i represents the utilization time of these resources, PD_i indicates the computational cost of DC v_i, and C_i denotes the required computational capacity.

Another important objective in the SFC placement challenge is end-to-end latency since it directly affects SLA compliance and QoS satisfaction. Variations in SFC processing durations and possible network congestion might result from latency changes caused by VNF placement choices and the mapping of virtual links onto DC pathways. Propagation latency, virtualization latency, and computing latency add up to the end-to-end latency:

$$O_{E2E} = L_P + L_V + L_C \tag{16}$$

The time it takes for an SFC request to travel from the source to the destination is known as the propagation latency (L_P). The physical distance and signal propagation speed between servers determine this:

$$L_P = \sum_{u \in U} \sum_{v_i \in V_s} \sum_{\epsilon_j \in V_v^{(u)}} x_{i,j}^{(u)} . Z_{i,j}^{(u)} \tag{17}$$

where $Z_{i,j}^{(u)}$ denotes the signal propagation speed when VNF ϵ_j is deployed on DC v_i in request u.

Virtualization Latency (L_V) refers to the overhead caused by creating, configuring, and managing virtual resources in DCs, particularly during the initialization of VNF instances. The edge infrastructure's hardware capacity, resource availability, and VNF complexity all have an impact:

$$L_V = \sum_{u \in U} \sum_{v_i \in V_s} \sum_{\epsilon_j \in V_v^{(u)}} x_{i,j}^{(u)} . H_{i,j} \tag{18}$$

where $H_{i,j}$ represents the virtualization latency of deploying VNF ϵ_j on DC v_i, which depends on the VNF complexity and the hardware capabilities of the DC.

Computational Latency (L_C) results from the traversal of VNFs within an SFC and is obtained by summing the latencies between DCs to which the VNFs are mapped:

$$L_C = \sum_{u \in U} \sum_{\epsilon_{p,q} \in E_v} L_{p,q} . \varphi_j(v_p, \epsilon_j, v_q, \epsilon_{j+1}) . y_u \tag{19}$$

In the case of p-SFCs, the overall latency is determined by the critical path, defined as the path with the maximum cumulative latency among all possible paths.

Since network resources are commonly shared among multiple flows, ensuring fairness is as important as meeting the QoS requirements of individual flows [11]. Take a node with enough bandwidth and resource capacity C, for example. Two flows, SFC1 and SFC2, are processed by VNF instances ν_α and ν_β, both deployed on the same node. ν_α needs C units of resources to handle a single packet, but ν_β needs $2C$. Maximum throughput is reached if the system only accepts SFC1 and rejects SFC2 completely, however this allocation is obviously unfair and unfeasible. Admitting both flows and allocating resources as $C/3$ for SFC1 and $2C/3$ for SFC2 would be a more equitable strategy. This lowers the overall throughput while ensuring fairness. As a result, there is a basic trade-off between ensuring fairness and optimizing throughput, which needs to be controlled by careful resource allocation and flow control. When several VNF instances for the same network function are dispersed throughout several servers, the issue gets considerably more complicated, making it harder to strike a compromise between efficiency and justice. Let $\nu_u(t) = \sum_{v_i \in V_s} r_i^{(u)}(t),\ \forall u \in U$ and ν_u be the average throughput of SFC u [11]:

$$\nu_u = \lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T} \sum_{v_i \in V_s} r_i^{(u)}(t),\ \forall u \in U \tag{20}$$

where $r_i^{(u)}(t)$ is the input data rate v_i for post-processing request u at time t.

Mo and Walrand [18] define the utility of SFC u as follows:

$$O_{utility} = \frac{1}{|U|} \sum_{u\in U} \begin{cases} (1-\delta)^{-1} \nu_u^{(1-\delta)} & \text{if } \delta \neq 1 \\ \log \nu_u & \text{if } \delta = 1 \end{cases} \tag{21}$$

where $\delta \geq 0$ is a tuning parameter for fairness. A greater preference for fairness above throughput is shown by a larger δ in the strictly nondecreasing and concave utility function.

The optimization problem is finally defined. The optimization technique is reflected in the cumulative objective function, which strikes a balance between utility enhancement, latency reduction, cost efficiency, and service acceptance maximization:

$$\mathcal{F}_{obj} = \min_t \sum_{t\in\Delta t} \xi_{cost} \cdot O_{cost}(t) + \xi_{E2E} \cdot O_{E2E}(t) - \xi_{acc} \cdot O_{acc}(t) - \xi_{utility} \cdot O_{utility}(t) \tag{22}$$

where ξ_{cost}, ξ_{E2E}, ξ_{acc}, and $\xi_{utility}$ represent the relative importance of cost, latency, service acceptance, and utility, respectively.

4. Proposed model

This section presents the suggested FPPVS model to solve the issue of SFC placement under NFP in MEC situations. After describing the VNF instance sharing module, we start by summarizing the fundamentals of SFC parallelization. We then give a thorough description of how an FDRL-based algorithm for the placement and assembly of SFC requests is configured and trained. The model seeks to accomplish a balanced trade-off between four major objectives: utility improvement, delay minimization, cost efficiency, and service acceptance maximization. It is guided by the specified objective function.

4.1. SFC parallelism principle

Managing the deployment and execution of SFCs that users request is the primary duty of an NFV orchestrator. As shown in Fig. 1, s-SFCs can be converted into parallelized structures depicted as DAGs, which are made up of a primary chain and multiple auxiliary branch chains of VNFs, to improve efficiency and performance. At the egress node or an intermediary shaping function, each branch chain eventually rejoins the main chain.

Transforming an s-SFC into a p-SFC requires compliance with several principles [35]. In this process, each traffic flow is routed through the main chain, where it is processed by the VNFs deployed along it, while duplicate flows are simultaneously forwarded through the branch chains for complementary processing. The p-SFC's main chain must maintain the same modification vector in order to guarantee equivalency with the original s-SFC and the same functional results. Furthermore, while passive VNFs like monitors can function independently without compromising semantic integrity, dependency constraints among VNFs in the s-SFC must be strictly maintained in the parallelized structure. For example, a Packet Header Inspector (PHI) must always come before shaping functions or a Network Address Translator (NAT). This study makes the assumption that all dependency ties between VNFs are known prior to placement, in line with earlier research by Luo et al. [35].

Every VNF in an SFC can be categorized as either a shaper or a monitor. While shaper VNFs actively analyze and modify the traffic to impose particular network policies, monitor VNFs watch traffic flows without changing the packets. Branch chains with monitor or shaper VNFs eventually converge back into the main chain when building a p-SFC, either at the egress or at a shaper node. This structural arrangement guarantees that monitors offer visibility and analysis without compromising the integrity of the traffic, while shapers carry out the required flow changes. Before applying the SFC deployment and routing process, Algorithm 1 is applied for parallelization. The proposed SFC parallelization algorithm converts an s-SFC into a p-SFC consisting of a main chain and multiple branch chains while preserving VNF dependencies and ensuring processing equivalency. The algorithm begins by initializing the main chain with the ingress node, while the set of branch chains and outstanding monitor VNFs are kept empty. It then iterates through each VNF in the s-SFC: if the current VNF is a monitor, a new branch chain is created from the last VNF in the main chain to the monitor, which is also added to the outstanding set. If the VNF is a shaper, it is appended to the main chain, and any dependent monitors in the outstanding set are connected to it before being removed. At the end, the last VNF in the main chain is connected to the egress. While monitor VNFs are arranged into branches without changing flow semantics, this approach guarantees that shaper VNFs stay in order along the main chain, maintaining the modification vector of the original s-SFC. Furthermore, the technique guarantees that the final p-SFC retains complete functional correctness by rigorously maintaining dependency restrictions and removing superfluous placements.

The objective of the transformation is to convert an input s-SFC, denoted as $SFC_u = \{\nu_1, \nu_2, \ldots, \nu_n\}$, into a p-SFC, represented by the graph G_v. Along with SFC_u, the dependency matrix CD is also provided as input to guide the conversion from s-SFC to p-SFC. The goal is to construct the graph G_v, which consists of a p-SFC with a main chain $\mathbb{M}$ and a set of branch chains $\mathbb{B} = \{\mathbb{b}_1, \mathbb{b}_2, \ldots\}$. The main chain is initialized with the source VNF of the given SFC request (i.e., $\mathbb{M} \leftarrow \nu_{src}$). The parallelization process proceeds by iteratively processing each VNF. If $\nu_j \in SFC_u$ is a monitor VNF, a new branch chain is created, initialized with the last VNF in $\mathbb{M}$ and the current VNF ν_j. This branch is then added to $\mathbb{B}$:

$$\mathbb{b} = \{\mathbb{M}_{\text{last}}, \nu_j\},\ \mathbb{B} = \mathbb{B} \cup \{\mathbb{b}\} \tag{23}$$

where $\mathbb{M}_{\text{last}}$ denotes the most recent VNF in the main chain. To keep track of processed monitor VNFs, each ν_j of type monitor is added to the set Θ after processing, i.e., $\Theta = \Theta \cup \{\nu_j\}$.

If $\nu_j \in SFC_u$ is a shaper VNF, it is appended directly to the main chain:

$$\mathbb{M} = \mathbb{M} \cup \{\nu_j\} \tag{24}$$

For shaper VNFs, dependency checks with the set Θ must be performed to determine possible connections that allow ν_j to execute in parallel with monitor VNFs. The dependency function between ν_j and

Algorithm 1
s-SFC to p-SFC conversion process.

Input: an s-SFC: $SFC_u = \{v_1, v_2, \ldots, v_n\}$, dependency matrix CD, v_{src}, v_{dst}.
Output: a p-SFC as graph G_v with main chain $\mathbb{M}$ and branch chains $\mathbb{B} = \{\mathbb{b}_1, \mathbb{b}_2, \ldots\}$.

$\mathbb{M} \leftarrow v_{src}$; // Initialize the main chain with the source VNF
$\mathbb{B} = \{\}$; // Initialize the set of branch chains
$\Theta = \{\}$; // Initialize the set of outstanding monitor VNFs
foreach $v_j \in SFC_u$ **do**
  if v_j is a monitor type VNF **then**
    $\mathbb{b} = \{\mathbb{M}_{\text{last}}, v_j\}$; // Create a new branch chain
    $\mathbb{B} = \mathbb{B} \cup \{\mathbb{b}\}$; // Add the branch chain to $\mathbb{B}$
    $\Theta = \Theta \cup \{v_j\}$; // Add v_j to Θ
  end
  if v_j is a shaper type VNF **then**
    $\mathbb{M} = \mathbb{M} \cup \{v_j\}$; // Append v_j to the main chain
    foreach $q_i \in \Theta$ **do**
      if $\text{Dep}(v_j, q_i) = 1$ **then** // v_j depends on q_i
        $v_j \rightarrow q_i$; // Extend the branch ending at q_i to v_j
        Remove q_i from Θ to prevent reuse.
      end
    end
  end
end
$\mathbb{M}_{\text{last}} \leftarrow v_{dst}$; // Connect the last main-chain VNF to the destination
Create graph G_v using $\mathbb{M}$ and $\mathbb{B}$.
return G_v;

$\mathscr{g} \in \boldsymbol{\Theta}$ is formulated as:

$$\textbf{Dep}(\mathscr{r}_j, \mathscr{g}) = \begin{cases} 1 & \textbf{if } \boldsymbol{cd}_{j,\mathscr{g}} \neq 0 \\ 0 & \textbf{otherwise} \end{cases} \tag{25}$$

where $\textbf{Dep}(\mathscr{r}_j, \mathscr{g}) = 1$ indicates a dependency between $\mathscr{r}_j$ and $\mathscr{g}$, implying that an existing branch ending at $\mathscr{g}$ should be extended to $\mathscr{r}_j$ (i.e., $\mathscr{r}_j \rightarrow \mathscr{g}$). Since q may no longer be reused for future connections, it is removed from the set $\boldsymbol{\Theta}$.

This process continues for all $\mathscr{r}_j \in \boldsymbol{SFC_u}$. Finally, the last VNF in the main chain is connected to the destination VNF of the SFC (i.e., $\mathbb{M}_{\textbf{last}} \leftarrow \mathscr{r}_{\boldsymbol{dst}}$), thereby completing the parallelization process. In the proposed model, the main chain of the p-SFC, together with each of its branch chains, is treated as an independent sub-SFC. Each sub-SFC is routed separately through the network, allowing parallel and isolated processing of traffic flows.

By retaining all shaper VNFs within the main chain of the p-SFC and preserving their original order from the s-SFC, this algorithm ensures that the modification vector remains unchanged, thereby satisfying the principle of parallel processing equivalency. At the same time, this preservation of order guarantees that the dependency relationships among shaper VNFs are maintained, thus fully meeting the VNF dependency preservation principle. Specifically, the p-SFC preserves all original dependencies: a shaper VNF depending on another shaper maintains the same order in the main chain, a monitor depending on a shaper is attached via a new branch from that shaper, and a shaper depending on a monitor is directly connected to it, while monitors never depend on each other.

4.2. VNF sharing

A VNF can be deployed in two alternative ways: by instantiating a new instance or by sharing an already initialized one [9]. Several prior works approach VNF placement by directly assigning each VNF to a node with the exact number of resources it demands. Precise resource allocation is often impractical under resource scarcity, and adding more VNFs can increase hop-counts in the MEC topology, reducing placement efficiency. Whether to reuse an existing instance and which instance to choose are two crucial choices for effective reuse. Predicting future VNF requests is much more difficult with online SFC placement since each allocation affects future SFC routing. Thus, a crucial and difficult part of effective online SFC placement is figuring out when and when to reuse VNF instances.

Inspired by [5,13,42,47], FPPVS integrates an instance sharing module that decides how VNFs are deployed in order to maximize resource consumption. In order to help the agent choose suitable hosts for freshly initiated instances, FPPVS uses a queuing network to record the distribution of VNFs. An LSTM module and a SQN memory make up the SFC Queue Network (SQN), which is intended to record and forecast the temporal distribution of necessary VNFs. In the meantime, the time-depth (TD) option determines the length of the SQN memory, which is organized as a tiered queue that holds previously needed VNFs. A trade-off between minimizing computing cost and capturing adequate temporal correlations is used to choose the TD parameter in the queuing network.

The SQN memory and an LSTM neural network are the two main parts of the SQN's detailed design, which is shown in Fig. 2. The SQN memory is organized as a layered queue, where each layer represents the computing resources required by a specific SFC request $\boldsymbol{u}$, denoted as $\boldsymbol{\varphi_u}$. In addition, details of the physical nodes hosting the VNFs and their

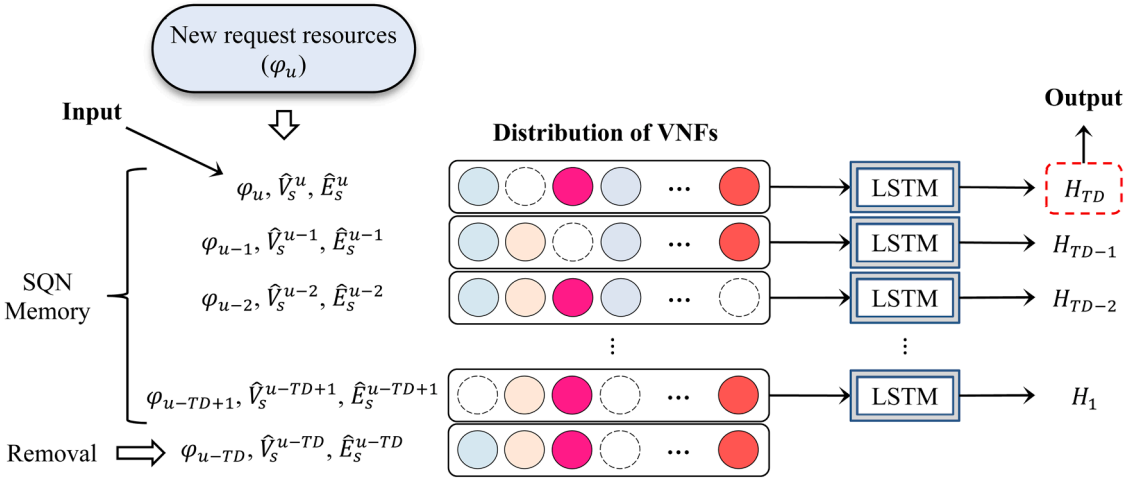


Fig. 2. Structure of the SQN with memory layers and LSTM for predicting VNF demand distribution.

associated links are retained within the queue memory and incorporated as inputs for training the LSTM. Specifically, $\hat{\boldsymbol{V}}_{\boldsymbol{s}}^{\boldsymbol{u}}$ represents the set of nodes hosting the VNFs associated with $\boldsymbol{\varphi_u}$, while $\hat{\boldsymbol{E}}_{\boldsymbol{s}}^{\boldsymbol{u}}$ denotes the set of links supporting the VNFs for the same request. This information allows the LSTM to capture both computational and network resource distributions for more accurate prediction and decision-making.

As new requests arrive over time, the resource requirements of each accepted SFC are sequentially recorded in this memory. This historical record enables the system to capture temporal patterns in VNF usage. The current system state in FDRL, $\mathcal{s}_\tau$, serves as the input to the SQN. The operational process of SQN, as detailed in Algorithm 2, involves two main stages: memory updating and relationship extraction. In lines 1–8, when a new state $\mathcal{s}_\tau$ is observed, SQN first checks whether a new SFC request $\boldsymbol{u}$ has arrived. If no new request exists, the memory remains unchanged. Otherwise, the SQN extracts the required computing resources $\boldsymbol{\varphi_u}$ from $\mathcal{s}_\tau$. To maintain a bounded memory, the oldest layer of the queue (i.e., $\boldsymbol{\varphi_{u-TD}}$) is discarded, and the new resource vector $\boldsymbol{\varphi_u}$ is added to the SQN memory, ensuring the memory reflects the most recent TD requests. Meanwhile, lines 8–11 correspond to the execution of the LSTM network and the extraction of its output.

When a placement decision must be made for the current state $\mathcal{s}_\tau$, the SQN leverages the LSTM to model temporal dependencies in the sequence of past VNF requirements. The input to the LSTM is a sequential vector representing the queued resource demands from the most recent TD requests, arranged in temporal order: $[\boldsymbol{\varphi_{u-TD+1}}, \boldsymbol{\varphi_{u-TD+1}}, \ldots, \boldsymbol{\varphi_{u-1}}, \boldsymbol{\varphi_u}]$. This input allows the LSTM to capture patterns and correlations across successive requests. The output of the last LSTM layer, $\boldsymbol{H_{TD}}$, provides a predicted distribution of the required VNFs, which informs the agent about likely resource needs in the current state. Rather than retraining from scratch, the LSTM updates gradually in dynamic MEC situations. To capture temporal demand patterns, it encodes VNF types, normalizes inputs, and keeps recent SFC requests in a memory queue. The model can precisely forecast future VNF demands and enhance placement choices because it is trained using previous SFC requests and trace-driven simulations.

To handle conflicts when multiple SFCs request the same VNF instance, FPPVS first tries to reuse an existing shared instance. If several instances are available, it selects the one with the highest remaining capacity and the lowest communication cost. The request is accepted only if sufficient processor, memory, and bandwidth are available. Otherwise, another shared instance is considered. If no suitable instance exists, a new VNF instance is deployed. If resources are still insufficient, the request is rejected. In addition, the SQN-LSTM prediction module distributes VNF instances across MEC servers to reduce future contention, while the fairness-aware policy prevents a few SFCs from monopolizing shared resources. This helps balance the load and improve resource utilization.

Algorithm 2
SQN processing.

Input: Current system state ($\mathcal{s}_\tau$), SQN memory, time-depth (TD), LSTM unit.
Output: Predicted distribution of the required VNFs (H_{TD}).

1: % Update Checking
2: **if** new SFC request u arrives in $\mathcal{s}_\tau$ **then**
3: Extract required VNF resources φ_u from $\mathcal{s}_\tau$;
4: **if** number of entries in SQN memory $\geq TD$ **then**
5: Remove $\varphi_{u-TD}, \hat{V}_s^{u-TD}, \hat{E}_s^{u-TD}$ from SQN Memory;
6: **end**
7: Push φ_u into SQN memory;
8: **end**
9: % Relationship Extraction
10: LSTM_Input ← $[\varphi_{u-TD+1}, \varphi_{u-TD+1}, \ldots, \varphi_{u-1}, \varphi_u]$;
11: H_{TD} ← LSTM (LSTM_Input);
12: **return** H_{TD};

4.3. FDRL details

The suggested FPPVS framework includes an FDRL module to deploy the generated sub-SFCs in parallel. Each sub-SFC's placement is designed as a Markov Decision Process (MDP) [47], allowing the agent to continuously interact with the network environment and discover the best placement strategy. Convolutional Neural Networks (CNNs) are used to implement both the actor and the critic in the FDRL agent's actor-critic architecture. A fully connected layer with 256 neurons comes after three convolutional layers with a kernel size of 3×3, each of which has 64 filters and ReLU (Rectified Linear Unit) activation functions. While the critic network assesses the corresponding state values to direct policy optimization, the actor network outputs the probability distribution over possible placement actions. Batch normalization is used following each convolutional layer to increase training stability and speed convergence. Additionally, a dropout layer with a rate of 0.3 is used to reduce overfitting and improve the learnt model's capacity for generalization. Mini-batches of 128 samples are used for gradient updates during the learning process, and the networks are trained using the Adam optimizer with a learning rate of 0.0005. Fig. 3 depicts the actor-critic-based FDRL algorithm's design.

The corresponding FDRL agent interacts with the environment according to the tuple $\langle \mathscr{S}, \mathscr{A}, \mathscr{R}, \boldsymbol{\pi} \rangle$, where $\mathscr{S}$ denotes the state space, $\mathscr{A}$ the action space, $\mathscr{R}$ the reward function, and $\boldsymbol{\pi}$ the policy. The individual components of the MDP are described in the following subsections.

State: At choice step $\boldsymbol{\tau}$, the FDRL operator watches the environment through a state $\mathscr{s}_{\tau} \in \mathscr{S}$, spoken to as a fixed-length include vector rather than chart embeddings. The state incorporates the current sub-SFC (unique and assistant VNFs), candidate hubs and their accessible assets, sub-SFC properties (entry time, TTL, activity request, and QoS prerequisites), as well as the reliance framework $\boldsymbol{CD}$ and choice parameter set $\boldsymbol{\Psi}$. Hence, the state is characterized as:

$$\mathscr{s}_{\tau} = \{\langle \boldsymbol{G_v}, \boldsymbol{t_{arr}}, \boldsymbol{t_{ttl}}, \boldsymbol{r_{\Delta}}, \mathscr{l}, \mathscr{P}, \boldsymbol{v_{src}}, \boldsymbol{v_{dst}} \rangle, \langle (\boldsymbol{D_j}, \boldsymbol{L_j}), \ \forall \boldsymbol{j} = 1, 2, \ldots, |\boldsymbol{V_v}| \rangle, \langle \boldsymbol{v_{cur}} \in \boldsymbol{V_s}, \boldsymbol{z} \in \boldsymbol{\Gamma}(\boldsymbol{v_{cur}}), \boldsymbol{v_{cad}}, \boldsymbol{G_s} \rangle, \langle \boldsymbol{CD}, \boldsymbol{\Psi} \rangle\} \tag{26}$$

where $\boldsymbol{v_{cur}}$ signifies the VNF right now being put and $\boldsymbol{\Gamma}(\boldsymbol{v_{cur}})$ speaks to its candidate hubs. Continuous features are normalized, categorical features are one-hot encoded, and all features are concatenated into a fixed-length vector for input to the CNN-based FDRL model.

Action: The agent first applies resource-aware pruning to narrow down the candidate nodes. Nodes that do not have enough computing resources or available bandwidth are excluded from further consideration. Next, among the feasible nodes, we leverage the predicted VNF demand distribution from the SQN-LSTM module to prioritize nodes that have historically been favorable for similar VNFs. This two-level filtering substantially reduces the combinatorial explosion. Moreover, instead of checking all possible node combinations, the agent places VNFs one by one by selecting a deployment node for the current VNF and a feasible neighboring node for the next VNF. Meanwhile, this mechanism also enables the reuse of previously deployed VNF instances. For further efficiency, softmax-based action sampling is adopted to bias the policy toward higher-probability actions while still exploring alternative choices.

Candidate actions include reusing existing VNF instances or deploying new instances on candidate nodes selected according to the predicted distribution $\boldsymbol{H_{TD}}$. At each time step $\boldsymbol{\tau}$, the local agent evaluates all candidate actions by computing their Q-values using its Q-network, $\boldsymbol{Q}(\mathscr{s}_{\tau}, \mathscr{a}_{\boldsymbol{k}})$, where $\mathscr{a}_{\boldsymbol{k}} \in \boldsymbol{v_{cad}}$. The Q-values are then converted into action selection probabilities using the softmax function [13]:

$$\boldsymbol{P}(\mathscr{s}_{\tau} | \mathscr{a}_{\boldsymbol{k}}) = \frac{\exp(\boldsymbol{Q}(\mathscr{s}_{\tau}, \mathscr{a}_{\boldsymbol{k}})/\boldsymbol{\eta})}{\sum_{l} \exp(\boldsymbol{Q}(\mathscr{s}_{\tau}, \mathscr{a}_{l})/\boldsymbol{\eta})} \tag{27}$$

where $\boldsymbol{\eta}$ controls the trade-off between exploration and exploitation. An action $\mathscr{a}_{\boldsymbol{i}}$ is then randomly sampled according to the probability distribution $\boldsymbol{P}(\mathscr{s}_{\tau} | \mathscr{a}_{\boldsymbol{k}})$.

Reward: After each VNF placement, the agent receives a reward that reflects the quality of its decision and guides subsequent placement actions. A valid placement that satisfies all resource, latency, and dependency constraints receives a base reward of +1. To encourage VNF sharing and reduce instantiation overhead, an additional reward of +5 is assigned when the required VNF is deployed by reusing an existing instance. Otherwise, if a new VNF instance must be created, a small penalty of −1 is applied to account for the additional deployment cost. If the selected node violates any resource, latency, or dependency constraint, the agent immediately receives a penalty of −10. After all, VNFs of an SFC have been processed, a final reward proportional to the objective function $\mathscr{T}_{\boldsymbol{obj}}$ is assigned if the SFC is successfully deployed. Otherwise, if the deployment fails, the agent receives a penalty equal to

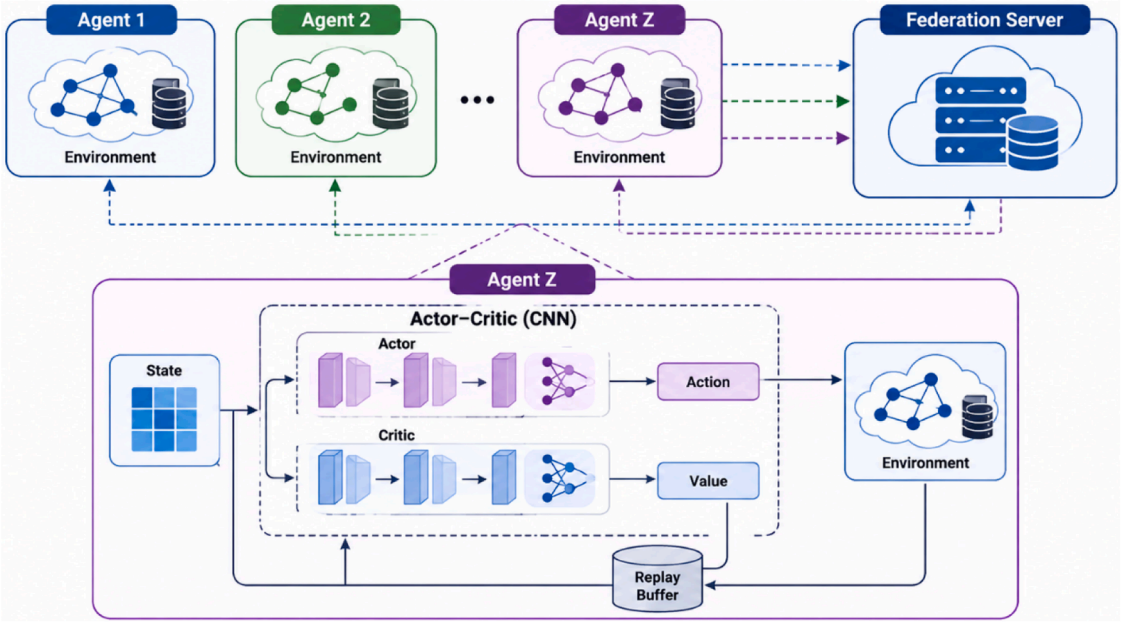


Fig. 3. Architecture of the FDRL algorithm based on actor-critic networks.

the accumulated reward magnitude stored in the memory buffer $\mathcal{M}_u$, discouraging infeasible placement decisions and promoting successful end-to-end SFC deployment. Accordingly, the reward function is defined as:

$$r_\tau(a_\tau, s_\tau) = \begin{cases} -10 & \textbf{if the node violates constraints}, \\ \lambda_s + 1 & \textbf{if the placement is valid}, \\ \mathcal{T}_{obj} & \textbf{if } y_u = 1, \\ -R_{max} & \textbf{if } y_u = 0, \end{cases} \tag{28}$$

where λ_s and R_{max} are defined as follows:

$$\lambda_s = \left\{ \begin{array}{ll} 5 & \textbf{if an existing VNF instance is reused}, \\ -1 & \textbf{otherwise}, \end{array} \right\} \tag{29}$$

$$R_{max} = \sum_{r_z \in \mathcal{M}_u} |r_z| \tag{30}$$

where HM_u denotes a memory buffer that records the cumulative reward history of SFC u.

Policy: The actor network receives the current state s_τ and outputs a probability distribution over all candidate actions. The agent then selects an action according to this distribution. The policy is defined as:

$$\pi_\theta(a_\tau \mid s_\tau) = \mathbf{Pr}\,(a_\tau \mid s_\tau; \theta) \tag{31}$$

where θ denotes the parameters of the actor network.

The critic network estimates the quality of each state–action pair by approximating the action-value function:

$$Q_\phi(s_\tau, a_\tau) \in (0, 1] \tag{32}$$

where ϕ represents the critic parameters. The critic provides feedback to the actor, allowing the policy to gradually improve during training.

The policy satisfies:

$$0 \le \pi_\theta(a_\tau \mid s_\tau) \le 1, \quad \sum_{a'_\tau \in v_{cad}} \pi_\theta(a_\tau \mid s_\tau) = 1 \tag{33}$$

where v_{cad} is the set of candidate actions for request u.

Finding the best course of action that maximizes the anticipated discounted cumulative reward is the actor's objective:

$$\pi^*_\theta = \arg\max_{\pi_\theta} \mathbb{E}\left[\sum_{\tau=0}^{\infty} \omega^\tau r_\tau\right] \tag{34}$$

where ω is the discount factor.

Using the predictions from the SQN to adaptively prioritize potential nodes, the actor network determines the action selection strategy throughout each learning phase. In the meantime, the critic network provides guidance that stabilizes and speeds up learning by assessing the short-term reward and estimating the long-term worth of each state. In practical terms, the critic outputs $Q_\tau(s_\tau, a_\tau)$, which represents the expected reward for the selected action, while the actor computes $\widehat{Q}_\tau(s_\tau, a_\tau)$, which represents the probability distribution over actions. The actor then makes adjustments to its policy based on the critic's assessment, making sure that chosen acts efficiently optimize long-term cumulative reward.

The proposed FDRL framework uses an actor–critic architecture. Each local agent has an actor network to select actions and a critic network to evaluate them. The agents learn independently by interacting with their own environments. From time to time, they send their model parameters to a central server. The server combines them using FedAvg (Federated Averaging) and sends the updated global model back to all agents. This helps the agents learn faster and achieve more stable performance. The local training is built on $Q_\tau(s_\tau, a_\tau)$, defined as:

$$Q_\tau(s_\tau, a_\tau; \theta) = \mathbb{E}\left[\sum_{k=\tau}^{\infty} \omega^{k-\tau} r_{\tau+k}(a_\tau, s_\tau) | s_\tau, a_\tau\right] \tag{35}$$

where θ denotes the CNN-based Q-network parameters. Both actor and critic networks are jointly used to iteratively train the action-value function $Q_\tau(s_\tau, a_\tau; \theta)$ with respect to the parameter set θ. To ensure accurate approximation of the optimal action-value function, a loss function $\mathcal{L}(\theta)$ is defined as follows:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(r_\tau + \omega \max_{a'_\tau} \widehat{Q}_\tau\left(s_{\tau+1}, a'_\tau; \widehat{\theta}(\tau+1)\right) - Q_\tau(s_\tau, a_\tau; \theta(\tau))\right)^2\right] \tag{36}$$

where $\mathcal{L}(\theta)$ measures the discrepancy between the predicted value and the target. The parameters θ and $\widehat{\theta}$ correspond to those of the actor network and the critic network, respectively. The critic network $\widehat{Q}_\tau$ is employed to compute the target reward associated with executing action a'_τ in the subsequent state $s_{\tau+1}$.

To improve training stability and accelerate convergence, a soft target update strategy is adopted, allowing the target network to gradually follow the prediction network instead of copying its parameters directly. In addition, a reward-based adaptive ε-greedy strategy dynamically adjusts the exploration rate according to the rewards obtained during training, enabling the agent to balance exploration and exploitation more effectively. Since the action space is continuous, exploration is further enhanced using an ε-logit perturbation mechanism, where Gaussian noise is added to the actor's logits before the softmax operation. This introduces controlled randomness into action selection while maintaining stable policy learning.

5. Performance evaluation

We do a comprehensive study including several facets of SFC placement and orchestration in order to evaluate the performance of the suggested approach. The simulation setup, network topologies, baseline DRL methods for comparison, ablation analysis of framework components, and simulation results comprise the five sections of this section.

5.1. Simulation setup

The simulation environment used to assess the suggested model is described in this section. In order to ensure compatibility with established benchmarks, computational capacity for both VNFs and edge servers are obtained from previous studies [9,47]. Node capacities are uniformly distributed between 10 and 20 units, while link bandwidths range from 100 to 200 units with increments of 20, and link latencies are uniformly set between 5 and 20 units [13,42]. The processing cost per unit of computational resource is assumed to be $0.001 per hour.

In this study, simulations are performed for 4000 SFC requests. Traffic demands for each SFC request follow a power-law distribution 5 to 30, reflecting the heavy-tailed nature of task loads in edge computing scenarios, where a small fraction of requests accounts for most of the load while the majority exhibit moderate demand [6,47]. Maximum tolerable latencies for requests are sampled uniformly within 100 to 150 ms, and initial packet sizes are drawn from a uniform distribution of 50 to 100 packets. Each SFC request is assigned a service lifetime sampled from an exponential distribution with a mean of 1000 time units, while SLA latency requirements are uniformly distributed between 2- and 4-time units. The auxiliary VNFs representing the source and destination of the SFC are independently mapped to randomly selected DCs [42,68]. Each SFC consists of 4 to 10 VNFs, selected from five distinct types—IDS, DPI, LB, FW, and NAT—each with heterogeneous resource demands [1,6]. The computational resources required for VNF instantiation follow a uniform distribution of 500–900 Mbps, processing latencies per data unit are set uniformly between 0.01 and 0.2 ms, and the per-link bandwidth requirement for each VNF ranges from 20 to 100 units.

Agents are trained in batch mode over 2000 iterations. In each iteration, the agent interacts with the environment over 10 episodes, each involving the placement of 20 SFCs, to populate its experience

buffer. The collected experiences are then used to update the agent's parameters in batch. Both the environment and buffer are reset after each iteration, reflecting an on-policy training methodology. The Adam optimizer is employed, with actor and critic learning rates set to 0.00001 and 0.0001, respectively [35]. All simulation modules and agents are implemented in MATLAB. Experiments are executed on a Windows 11 workstation equipped with a 4.1 GHz Intel Core i7 processor and 32 GB of RAM. Each experiment is repeated 25 times, and the reported results represent the average performance across these runs.

5.2. Network topologies

The proposed model is evaluated in two backbone and metropolitan network scenarios [9]. For the backbone case, we use the USAnet topology shown in Fig. 4 (a), consisting of 24 switches and 47 links. For the metropolitan scenario, we consider the access network of an Italian mobile operator, which interconnects 17 mobile traffic aggregation nodes to a Serving Gateway via 29 links, as illustrated in Fig. 4 (b). All nodes in both topologies are capable of deploying a VNF server.

5.3. Baselines methods

To evaluate the performance of the proposed FPPVS model, we selected four representative state-of-the-art methods as baselines: RVNF [40], DRL-SFCP [39], PVFP [41], and DPC [42]. RVNF focuses on DRL-based VNF reuse, DRL-SFCP leverages DRL with GCN and Seq2Seq models to optimize SFC placement for maximizing long-term revenue, PVFP emphasizes cost-efficient VNF placement, and DPC activates non-conflicting VNFs in parallel using A3C to maximize cumulative rewards and enhance resource utilization. These methods were chosen because they collectively cover a broad spectrum of SFC placement strategies, including DRL-based dynamic VNF placement, p-SFC execution, predictive VNF provisioning, and enhanced SFC placement heuristics. All baseline methods were either re-implemented according to their descriptions in the respective papers or executed using publicly available code to ensure correctness and reproducibility. Meanwhile, to guarantee a fair and consistent comparison, all experiments were conducted under identical assumptions, network topologies, and parameter configurations. This includes the same resource capacities, link characteristics, SFC request distributions, and simulation settings.

5.4. Ablation study

A sensitivity analysis was conducted to examine the effect of the objective function weights (ξ_{cost}, ξ_{E2E}, ξ_{acc}, and $\xi_{utility}$) on system performance. During the experiments, the sum of these weights was fixed at one ($\xi_{cost} + \xi_{E2E} + \xi_{acc} + \xi_{utility} = 1$). The results show that the best overall performance, in terms of balancing low latency, high service acceptance, and high throughput, is achieved when $\xi_{cost} = 0.15$, $\xi_{E2E} = 0.35$, $\xi_{acc} = 0.25$, and $\xi_{utility} = 0.25$. The same weight configuration consistently provides the best trade-off for both the Backbone and Metropolitan network topologies.

To evaluate the impact of the TD parameter on system performance in SQN-LSTM, we performed sensitivity analysis in both backbone and metropolitan topology environments. By examining the trade-off between LSTM prediction accuracy and the ensuing SFC placement performance in terms of latency (ms), service acceptance (%), and throughput (units), TD was empirically adjusted in both scenarios. Table 3 presents the analysis's findings. The findings verify that very small TD values (e.g., $TD \leq 3$) do not capture enough temporal relationships, leading to longer latencies and low prediction accuracy. On the other hand, model complexity and response overhead are increased by overly large TD values (e.g., $TD \geq 10$), which also deteriorates performance. The optimal balance is reached with an intermediate value of $TD = 5$, which produces the most acceptance and throughput in both network types together with the lowest latency. In particular, $TD = 5$ consistently yields the best trade-off for both metropolitan and backbone topologies. Additionally, the middle TD increases service acceptance

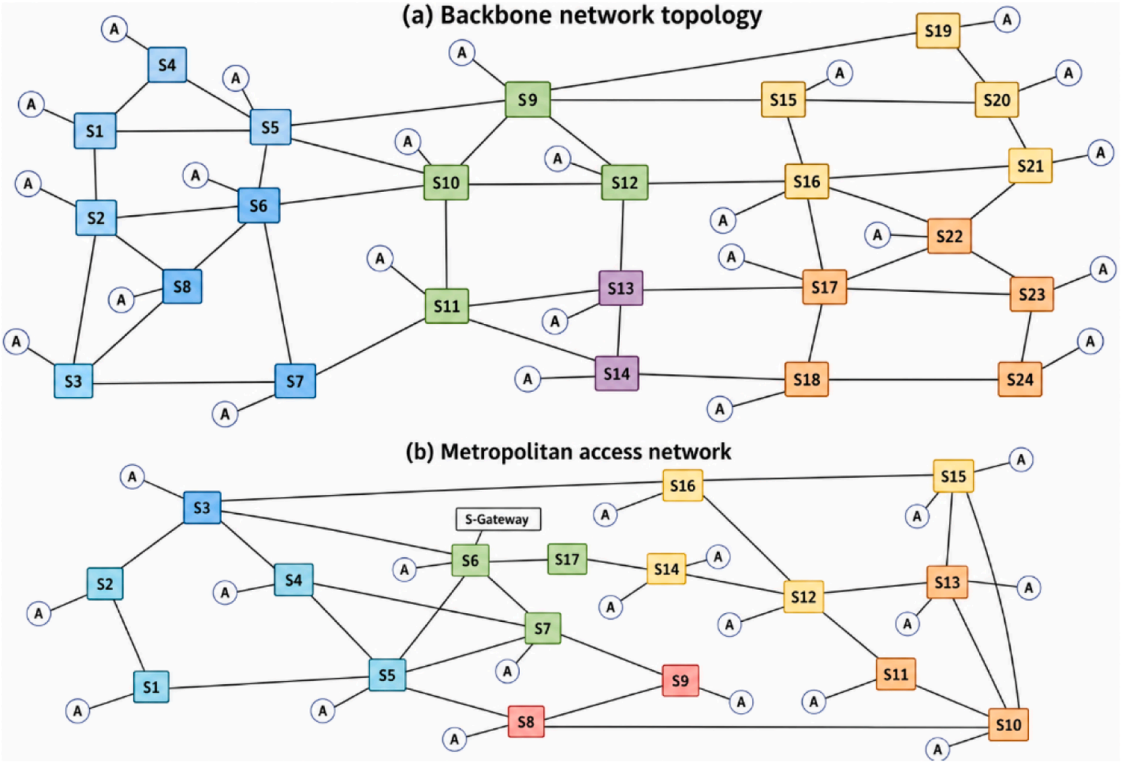


Fig. 4. (a) Backbone network topology (USAnet) and (b) metropolitan access network. The backbone network consists of 24 switches and 47 links, whereas the metropolitan access network comprises 17 aggregation nodes and 29 links. The symbols S and A denote switches and access points, respectively.

Table 3
Sensitivity analysis of TD parameter on system performance metrics for backbone and metropolitan topologies. The best values are bolded.

Time-depth (TD)	Backbone			Metropolitan		
	Service acceptance (%)	Latency (ms)	Throughput (units)	Service acceptance (%)	Latency (ms)	Throughput (units)
1	61.6 ± 0.8	169.1 ± 2.1	73.3 ± 1.8	58.8 ± 0.8	210.6 ± 3.0	71.6 ± 1.3
2	77.2 ± 0.8	152.7 ± 2.5	92.5 ± 1.6	64.0 ± 0.8	182.1 ± 2.9	88.0 ± 1.4
3	80.1 ± 1.1	144.1 ± 2.0	101.5 ± 2.0	72.6 ± 0.5	168.0 ± 2.8	95.0 ± 1.1
5	88.1 ± 0.8	134.4 ± 2.2	116.8 ± 1.6	75.9 ± 0.6	160.1 ± 2.6	97.6 ± 1.0
7	76.08 ± 1.2	136.2 ± 2.4	100.4 ± 2.6	69.79 ± 0.6	184.8 ± 3.3	81.8 ± 0.9
10	66.16 ± 0.7	148.0 ± 3.0	95.3 ± 1.9	64.42 ± 1.0	178.0 ± 3.7	74.6 ± 1.1
15	64.33 ± 0.8	155.4 ± 3.1	77.1 ± 1.5	60.35 ± 0.9	207.9 ± 3.5	69.7 ± 1.6

and throughput by around 37–42% and decreases latency by over 22% when compared to extreme values ($TD = 1$ or $TD = 15$). The findings show that optimizing the advantages of LSTM-based VNF demand prediction and guaranteeing effective SFC placement depend on proper TD tuning.

Ablation research was carried out by contrasting the entire FPPVS framework with a number of reduced versions in order to measure the contribution of each component in the suggested framework. FPP, which eliminates the VNF sharing module; FPVS, which disables SFC parallelization; FPPVS+Greedy, which substitutes a greedy strategy for the FDRL-based placement policy; PPVS, which eliminates the fairness-aware resource allocation mechanism; FDRL, which solely uses the reinforcement learning placement strategy and ignores other modules for deployment; and FDRL+VNF Sharing, FDRL+NFP, and FDRL+Fairness, which gradually incorporates the FDRL algorithm with one more module. Three important performance metrics—service acceptance ratio, average end-to-end latency, and network throughput—were used to assess both backbone and metropolitan network topologies. The results are summarized in Table 4, where the best values are highlighted in bold.

The complete FPPVS framework consistently achieves the best performance under both network topologies, demonstrating that the four modules complement each other. FPPVS improves throughput by 11.3%, lowers average latency by 9.7%, and raises the average service acceptance ratio by 7.2% when compared to the standard FDRL model, demonstrating that reinforcement learning alone is not enough to properly utilize network resources. SFC parallelization offers the most latency improvement among the separate improvements. The FDRL+NFP setup lowers the average latency by 7.1% as compared to FDRL, suggesting that running independent VNFs concurrently is a very efficient way to shorten the SFC processing route. The VNF sharing module primarily enhances the use of resources. By reusing existing VNF instances, FDRL+VNF Sharing reduces latency while increasing average service acceptance ratio by 3.3% and throughput by 5.0% when compared to FDRL. A balanced distribution of resources across conflicting requests is produced by the fairness-aware module. When compared to FDRL, FDRL+Fairness avoids resource monopolization under high traffic while increasing throughput by 6.2% and the average acceptance ratio by 4.0%. Performance is significantly reduced when a greedy placement technique is used in place of the FDRL agent. The full FPPVS enhances throughput by 5.6%, lowers latency by 7.5%, and improves average acceptance ratio by 4.5% when compared to FPPVS+Greedy, highlighting the significance of adaptive learning. Measurable performance degradation results from the removal of any one module. FPPVS enhances throughput by 3.5%, lowers latency by 4.3%, and improves the average acceptance ratio by 2.5% when compared to FPP, which inhibits VNF sharing. FPPVS achieves an average latency reduction of 8.0% when compared to FPVS, which eliminates SFC parallelization, demonstrating the significant contribution of NFP to delay reduction. Similarly, FPPVS increases throughput by 1.7%, lowers latency by 2.1%, and improves the average acceptance ratio by 1.3% when compared to PPVS, demonstrating that fairness-aware allocation further improves overall system efficiency. Finally, compared with the strongest partial configuration (FDRL+NFP), the complete FPPVS still achieves an average 1.6% higher service acceptance ratio, 2.5% lower latency, and 2.7% higher throughput. These results show that combining FDRL, VNF sharing, SFC parallelization, and fairness-aware resource allocation provides greater overall benefits than using any single component alone.

An additional experiment was conducted to compare the learning capability of different DRL algorithms using the expected cumulative reward. Since the proposed framework adopts an FDRL strategy, its performance was compared with four representative DRL methods, namely Deep Q-Network (DQN), Proximal Policy Optimization (PPO), A3C, and Soft Actor-Critic (SAC). Fig. 5 presents the cumulative rewards obtained over 4,000 consecutive SFC requests for both the Backbone and Metropolitan topologies. To improve readability, the reward curves were smoothed using a moving average with a window size of 100. FDRL consistently obtains the highest cumulative reward and converges more smoothly than the competing approaches, as Fig. 5 illustrates. In comparison to DQN, PPO, A3C, and SAC, FDRL typically increases the cumulative reward by 3.1%, 2.7%, 1.0%, and 0.93%, respectively.

To explicitly quantify fairness, the proposed framework further evaluates the resource allocation using Jain's Fairness Index (JFI):

$$J = \frac{\left(\sum_{u\in U} l_u\right)^2}{|U|\sum_{u\in U} l_u^2},\ 0 < J \leq 1 \tag{37}$$

where l_u is the average throughput for request u.

In another experiment, four exemplary DRL algorithms—DQN, PPO,

Table 4
Ablation study of FPPVS under Backbone and Metropolitan topologies.

Model	Backbone			Metropolitan		
	Service acceptance (%)	Latency (ms)	Throughput (units)	Service acceptance (%)	Latency (ms)	Throughput (units)
FDRL	80.5 ± 1.1	150.2 ± 3.5	104.1 ± 2.2	69.2 ± 0.9	177.8 ± 4.0	86.8 ± 1.8
FDRL + VNF Sharing	83.4 ± 1.0	144.6 ± 3.0	109.8 ± 2.0	71.8 ± 0.8	171.2 ± 3.5	91.5 ± 1.6
FDRL + NFP	86.5 ± 0.9	138.7 ± 2.6	113.4 ± 1.8	74.6 ± 0.7	164.2 ± 3.0	95.1 ± 1.3
FDRL + Fairness	84.6 ± 0.9	142.8 ± 2.8	111.1 ± 1.9	72.7 ± 0.8	169.5 ± 3.2	92.8 ± 1.5
FPPVS + Greedy	84.1 ± 1.0	145.5 ± 3.1	110.2 ± 2.0	72.3 ± 0.8	171.0 ± 3.4	91.9 ± 1.6
FPP (without sharing)	85.8 ± 0.9	140.6 ± 2.7	112.4 ± 1.8	73.8 ± 0.7	166.7 ± 3.1	94.2 ± 1.3
FPVS (without NFP)	84.7 ± 0.9	146.8 ± 3.2	110.9 ± 1.9	72.5 ± 0.8	173.6 ± 3.6	92.3 ± 1.5
PPVS (without fairness)	86.9 ± 0.8	137.9 ± 2.5	114.5 ± 1.7	74.9 ± 0.7	163.4 ± 2.9	96.0 ± 1.2
FPPVS	88.1 ± 0.8	134.4 ± 2.2	116.8 ± 1.6	75.9 ± 0.6	160.1 ± 2.6	97.6 ± 1.0

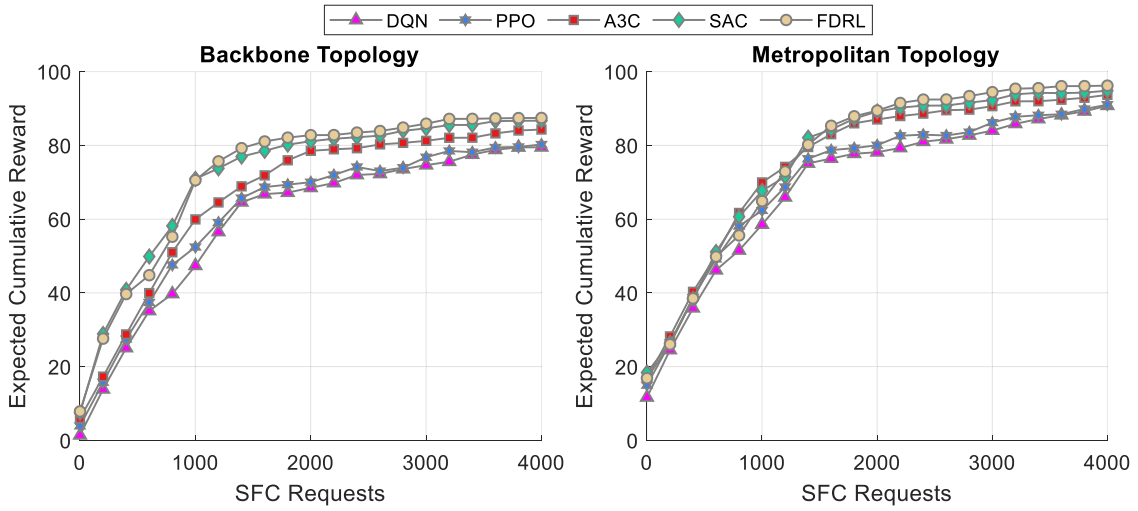


Fig. 5. Expected cumulative reward of DQN, PPO, A3C, SAC, and FDRL in Backbone and Metropolitan topologies.

A3C, and SAC—are compared with the FDRL-based FPPVS. The JFI is computed and the comparison is carried out following the processing of all SFC requests. A more equitable allocation of network resources among competing SFC demands is indicated by the higher values. Twenty separate runs of each algorithm were performed. The average JFI and standard deviation are presented as the findings. The best values are indicated in bold in Table 5, which summarizes the results. As demonstrated, in both network topologies, FPPVS consistently delivers the highest level of fairness. With an average JFI of 0.863, the suggested FPPVS outperforms DQN (0.813), PPO (0.827), A3C (0.842), and SAC (0.849) by 6.2%, 4.4%, 2.5%, and 1.7%, respectively. Similar improvements are observed in both backbone and metropolitan topologies, indicating that the proposed fairness-aware mechanism remains effective under different network scales.

In general, DQN is less suitable because it is designed for discrete action spaces and cannot effectively handle large SFC placement problems. PPO provides stable training but learns only from local experience. A3C converges faster, although its asynchronous updates may reduce stability. SAC explores efficiently but has higher computational cost and does not exploit knowledge from multiple domains. In contrast, FDRL combines actor–critic learning with federated model aggregation. It enables agents to learn locally while sharing knowledge across domains, resulting in faster convergence, better generalization, and more stable performance.

5.5. Simulation results

A number of cutting-edge techniques, such as RVNF, DRL-SFCP, PVFP, and DPC, are compared to the suggested FPPVS framework. Service acceptance, end-to-end latency, throughput, resource utilization, SFC assembly length, and runtime are the six main measures used to evaluate performance. The provided results are averaged over all SFC inquiries for a fair comparison. Tables 6 and 7 provide comprehensive performance comparisons for the metropolitan and backbone topologies, respectively. According to the comparison analysis, FPPVS constantly performs better than the current methods. Even while RVNF achieves a little faster execution time, the difference is negligible and has little bearing on real-world implementation. The integration of sophisticated elements like VNF instance sharing, SQN-based memory management, and LSTM-based prediction results in a slight runtime increase for FPPVS. By adding SQN-LSTM prediction and fairness-aware resource allocation, FPPVS further enhances placement judgments as compared to PVFP, which also uses federated DRL and SFC parallelization. Although PVFP claims a somewhat shorter end-to-end latency, this benefit comes at the expense of increased resource usage and

Table 5
Comparison of JFI for different DRL algorithms after processing all SFC requests.

Method	Backbone Topology	Metropolitan Topology	Average JFI
DQN	0.824 ± 0.011	0.801 ± 0.013	0.813
PPO	0.838 ± 0.010	0.816 ± 0.011	0.827
A3C	0.851 ± 0.009	0.832 ± 0.010	0.842
SAC	0.858 ± 0.008	0.839 ± 0.009	0.849
FDRL	0.872 ± 0.007	0.853 ± 0.008	0.863

Table 6
Quantitative performance comparison between FPPVS and competing methods in the backbone topology.

Evaluation metric	RVNF	DRL-SFCP	PVFP	DPC	FPPVS
Service acceptance (%)	85.4 ± 0.7	83.6 ± 0.8	86.9 ± 0.6	84.5 ± 0.7	88.1 ± 0.8
Latency (ms)	137.2 ± 2.5	141.1 ± 2.1	131.3 ± 2.3	139.4 ± 2.4	134.4 ± 2.2
Throughput (units)	112.5 ± 1.8	105.8 ± 1.5	115.0 ± 1.6	109.7 ± 1.7	116.8 ± 1.6
Resource utilization (%)	71.0 ± 2.5	69.4 ± 2.7	73.1 ± 2.2	70.2 ± 2.3	75.35 ± 1.9
SFC assembly length (hop)	4.78 ± 0.6	5.12 ± 0.7	4.69 ± 0.5	4.93 ± 0.6	4.63 ± 0.8
Execution time (s)	30.9 ± 2.1	40.1 ± 1.6	36.8 ± 1.7	42.7 ± 1.8	32.8 ± 2.1

Table 7
Quantitative performance comparison between FPPVS and competing methods in the metropolitan topology.

Evaluation metric	RVNF	DRL-SFCP	PVFP	DPC	FPPVS
Service acceptance (%)	73.7 ± 0.7	71.5 ± 0.7	75.2 ± 0.7	72.8 ± 0.6	75.9 ± 0.6
Latency (ms)	161.0 ± 2.3	166.8 ± 2.1	158.7 ± 2.4	165.3 ± 2.0	160.1 ± 2.6
Throughput (units)	95.1 ± 1.8	92.9 ± 1.5	96.8 ± 1.4	94.4 ± 1.6	97.6 ± 1.0
Resource utilization (%)	66.8 ± 2.4	64.8 ± 2.0	68.2 ± 2.2	65.8 ± 2.1	72.36 ± 1.2
SFC assembly length (hop)	5.26 ± 0.6	5.48 ± 0.7	5.10 ± 0.5	5.36 ± 0.6	4.77 ± 0.7
Execution time (s)	41.3 ± 2.2	50.9 ± 2.0	45.8 ± 1.5	53.6 ± 1.3	46.4 ± 1.8

decreased deployment effectiveness.

Across both the backbone and metropolitan topologies, FPPVS consistently achieves the best overall performance. On average, it improves service acceptance by 3.1%, 5.8%, 1.2%, and 4.3% compared with RVNF, DRL-SFCP, PVFP, and DPC, respectively. Similarly, the average throughput increases by 3.2%, 7.8%, 1.2%, and 5.0%, while resource utilization improves by 7.2%, 10.2%, 4.6%, and 8.7%, respectively. FPPVS also produces the shortest SFC assembly length, reducing the average path length by 6.2%, 11.3%, 3.9%, and 8.6% compared with the four baseline methods. In terms of end-to-end latency, FPPVS reduces latency by 1.3%, 4.4%, and 3.4% relative to RVNF, DRL-SFCP, and DPC, respectively, while remaining only 1.6% higher than PVFP, which achieves the lowest latency. Regarding execution time, FPPVS requires an average of 9.2% more runtime than RVNF because of the additional computations introduced by VNF sharing, fairness-aware scheduling, and SQN-LSTM prediction. However, it is still 13.5%, 4.8%, and 18.3% faster than DRL-SFCP, PVFP, and DPC, respectively. Overall, these results indicate that FPPVS provides the best trade-off between efficiency and deployment quality.

The additional complexity introduced by FPPVS is justified by its overall performance gains. Although the complete framework requires only a 9.2% longer execution time than RVNF, it consistently achieves higher service acceptance, better resource utilization, higher throughput, and shorter SFC assembly paths across both network topologies. Furthermore, its execution time remains lower than that of DRL-SFCP, PVFP, and DPC. It is worth noting that the reported execution time includes the computational overhead introduced by the FDRL framework, including local model training, periodic model aggregation, synchronization, and parameter exchange with the central server. Since federated aggregation is performed periodically rather than after every placement decision, and only model parameters are exchanged instead of raw data, the communication and synchronization overhead remains limited. Consequently, the additional cost introduced by federated learning is modest and is compensated by the improvements in scalability and placement quality.

Fig. 6 reports the results when SFC requests arrive sequentially, whereas Fig. 7 shows the cumulative rewards when requests are processed in batches of 100 concurrent SFC requests. The request processing approach is the primary cause of the various reward trends. The network still has enough resources when there are few SFC requests, and there is little rivalry amongst requests. Consequently, all approaches yield comparable outcomes. For instance, Fig. 6 shows that PVFP's performance is fairly similar to FPPVS's before about 500 queries. The performance disparity becomes increasingly noticeable as the demand grows. Because FPPVS blends fairness-aware resource allocation with VNF instance sharing, it retains larger cumulative rewards. These two strategies balance resource utilization, cut down on pointless deployments, and maintain a steadier learning process even in the face of high traffic. RVNF and DRL-SFCP, on the other hand, often create new VNF instances when resources are available and primarily rely on shortest-path placement. While this approach works rather well at low traffic loads, as the network gets more crowded, it eventually results in resource fragmentation and decreased rewards. Additionally, DPC receives less cumulative rewards, particularly in the early stages of learning. Its heuristic approach prioritizes lowering resource and energy consumption, which restricts throughput and lowers reward increase. Because both approaches are based on FDRL, the comparison with PVFP is very intriguing. Nevertheless, PVFP continues to generate new VNF instances more frequently, which raises deployment overhead.

We also evaluate how the DRL-based methods learn over time. Fig. 8 (a) and 8(b) show the average cumulative reward during training for the backbone and metropolitan topologies, respectively. The reward at each episode reflects how well the agent improves its placement policy. As training continues, higher rewards indicate better decision-making and learning performance. For clearer visualization, all learning curves are smoothed using a moving window of 50 episodes. These curves provide information about how well each algorithm adjusts its strategy over time. A consistent rise in rewards is shown across all approaches, indicating effective learning and convergence. Throughout training, FPPVS shows the most steady and continuous improvement, starting with a relatively high reward. In the end, it achieves the largest average reward since it converges faster than the other approaches. DRL-SFCP and DPC, on the other hand, exhibit a slower, more gradual increase, with early-stage rewards falling behind and eventually converging to slightly lower average rewards. This suggests that while these approaches are capable of learning efficient policies, their parallel action mechanisms may impede effective coordination and slow convergence. After the first few episodes, RVNF and PVFP show comparatively quick learning; despite a slight performance decline around episode 220, both rebound swiftly and by the end of training are on par with FPPVS.

In the final experiment, learning efficiency per episode was evaluated by looking at the training loss of the evaluated DRL-based approaches, taking into account both actor and critic losses during deployment. PVFP is not included in this comparison because it is based on the DQN algorithm. The suggested FPPVS model is directly equivalent to algorithms that use the actor-critic architecture, such as RVNF, DRL-SFCP, and DPC. Fig. 9(a) shows the average actor loss throughout training for both backbone and metropolitan topologies, while Fig. 9(b)

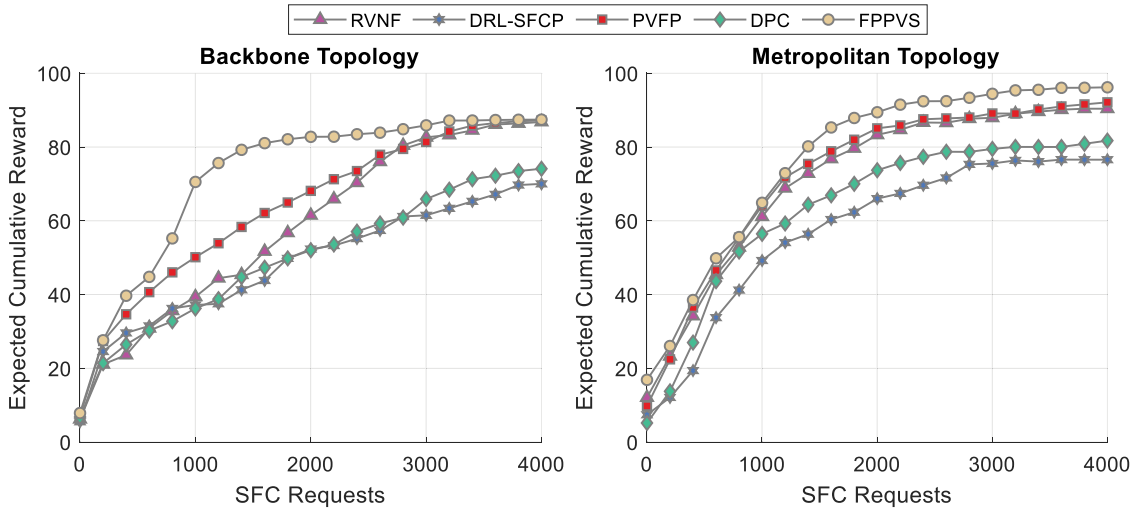


Fig. 6. Comparison of the average cumulative reward achieved by different methods during sequential SFC request processing.

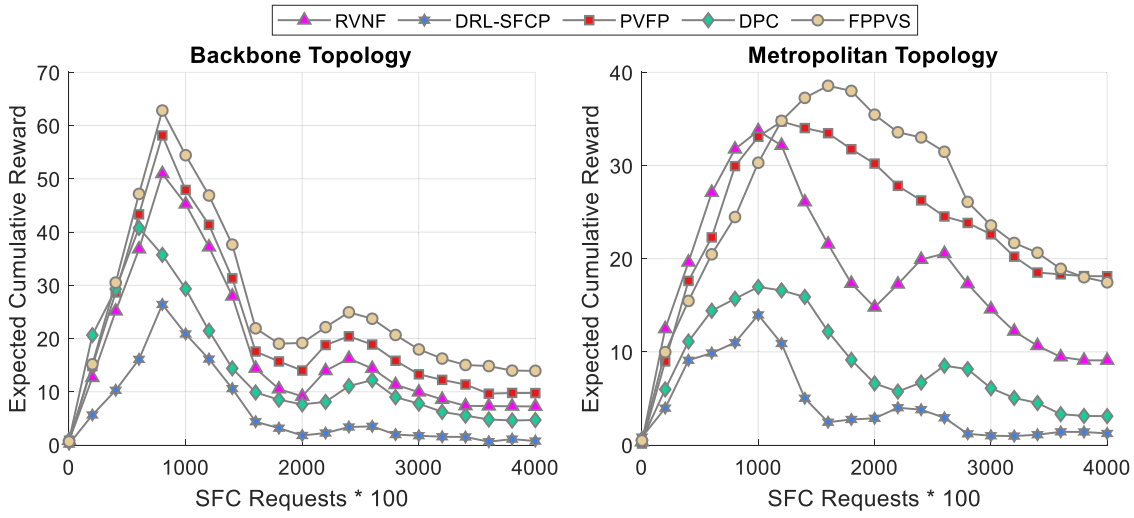


Fig. 7. Comparison of the average cumulative reward achieved by different methods over 100 consecutive SFC requests.

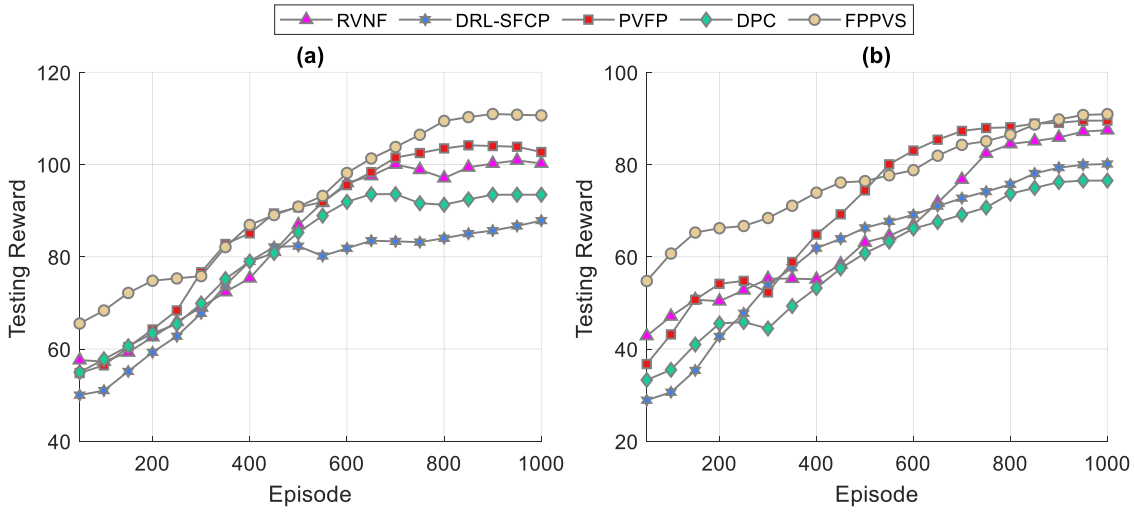


Fig. 8. Testing rewards of different DRL-based algorithms over training episodes for backbone and metropolitan topologies.

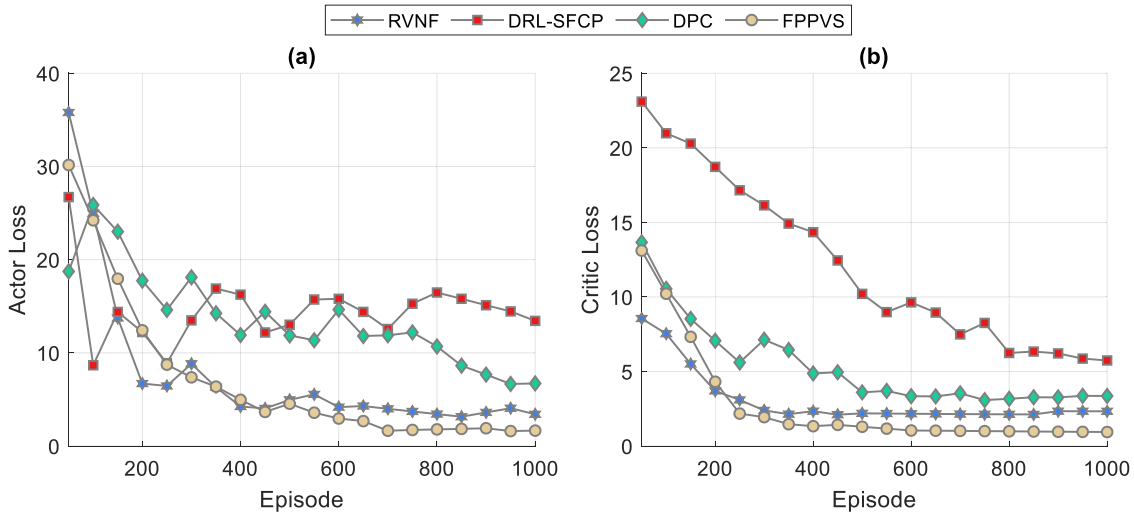


Fig. 9. Actor and critic loss curves during training of DRL-based methods, averaged over backbone and metropolitan topologies.

shows the corresponding critic loss. A window size of 50 is used to flatten the curves for better clarity. RVNF and DPC efficiently converge

about episode 600, indicating effective learning, according to our analysis of Fig. 9(a). Because of its more intricate architecture, DRL-SFCP exhibits greater instability at first but eventually reaches convergence by the conclusion of training. On the other hand, FPPVS achieves smoother and faster convergence around episode 500 thanks to its FDRL-based design. These patterns are further illustrated in Fig. 9(b), where DPC exhibits slight instability but stays at a low loss level, whereas RVNF retains little variation and converges close to episode 300. Around episode 800, DRL-SFCP reaches convergence after starting with a significant critic loss that progressively drops. By converging at episode 300, FPPVS exhibits the fastest and most reliable critic loss reduction, improving actor performance through more precise state value predictions.

The proposed framework does not improve performance by simply deploying more VNF instances. When an SFC is parallelized, the agent first tries to reuse existing VNF instances. A new instance is created only if no suitable shared instance is available. This keeps the overhead of parallelization low. The reward function also encourages VNF sharing and discourages unnecessary instantiations. In addition, the placement policy considers the available resources in each domain and avoids unnecessary traffic and inter-domain communication. As a result, FPPVS reduces end-to-end latency while using fewer VNF instances and less bandwidth. These gains come from smarter resource management and VNF reuse, not from creating extra network resources.

6. Conclusions

This paper proposed FPPVS, a fairness-aware framework for efficient SFC placement in MEC-enabled IoT networks. The framework combines SFC parallelization, VNF instance sharing, and federated deep reinforcement learning to improve deployment efficiency. The parallelization module converts sequential SFCs into partially parallel structures, allowing independent VNFs to run simultaneously and reducing end-to-end latency. The VNF sharing module reuses existing instances whenever possible and employs LSTM-based demand prediction to reduce deployment overhead and improve resource utilization. A fairness-aware allocation strategy balances network resources among competing SFC requests while maintaining QoS requirements. The FDRL-based placement engine enables distributed agents to learn collaboratively and adapt to changing network conditions. Experimental results show that FPPVS consistently improves service acceptance, resource utilization, throughput, and fairness while reducing latency compared with existing methods. Future work will investigate energy-aware placement, collaborative optimization with caching and task offloading, and more robust deployment under highly dynamic network conditions.

Declarations

Ethics approval

Not applicable.

Funding

Not applicable.

CRediT authorship contribution statement

Qiang Zhu: Writing – review & editing, Visualization, Validation, Resources, Methodology, Formal analysis. **MingZhu Li:** Writing – review & editing, Validation, Resources, Methodology. **Amin Rezaeipanah:** Writing – review & editing, Writing – original draft, Resources, Methodology, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This article is a research outcome of the 2025 Hunan Provincial Department of Education Scientific Research Project “Research on Tail Quantile Delay Modeling and Hierarchical Optimization Methods for SFC-DAG” (Project No.: 25C1631).

Data availability

No data was used for the research described in the article.

References

[1] W. Xu, J. An, H. Li, L. Gan, C. Yuen, Algorithm-unrolling-based distributed optimization for RIS-assisted cell-free networks, IEEe Internet Things J. 11 (1) (2023) 944–957.
[2] G. Sun, G. Zhu, D. Liao, H. Yu, X. Du, M. Guizani, Cost-efficient service function chain orchestration for low-latency applications in NFV networks, IEEe Syst. J. 13 (4) (2018) 3877–3888.
[3] G. Sun, Y. Li, D. Liao, V. Chang, Service function chain orchestration across multiple domains: a full mesh aggregation approach, IEEE Trans. Netw. Serv. Manag. 15 (3) (2018) 1175–1191.
[4] M. Dai, G. Sun, H. Yu, D. Niyato, Maximize the long-term average revenue of network slice provider via admission control among heterogeneous slices, IEEE/ACM Trans. Netw. 32 (1) (2023) 745–760.
[5] S. Zhang, W. Jia, Z. Tang, J. Lou, W. Zhao, Efficient instance reuse approach for service function chain placement in mobile edge computing, Comput. Netw. 211 (2022) 109010.
[6] T.W. Kuo, B.H. Liou, K.C.J. Lin, M.J. Tsai, Deploying chains of virtual network functions: on the relation between link and server usage, IEEE/ACM Trans. Netw. 26 (4) (2018) 1562–1576.
[7] H. Jiang, J. Guo, Z. Xiao, K. Yang, T. Li, B. Li, Collaborative perception and computing offloading in 6G air-ground integrated networks, IEEE J. Sel. Areas Commun. 44 (2025) 2821–2838.
[8] Z. Zhou, R. Yin, W. Wang, Q. Ni, C.C. Zarakovitis, Wireless deep mutual learning: challenges and opportunities, IEEE Commun. Mag. 64 (6) (2026) 88–94.
[9] V. Eramo, T. Catena, Application of an innovative convolutional/LSTM neural network for computing resource allocation in NFV network architectures, IEEE Trans. Netw. Serv. Manag. 19 (3) (2022) 2929–2943.
[10] Y. Peng, Y. Zhao, J. Dong, J. Hu, Adaptive opinion dynamics over community networks when agents cannot express opinions freely, Neurocomputing 618 (2025) 129123.
[11] L. Gu, D. Zeng, S. Tao, S. Guo, H. Jin, A.Y. Zomaya, W. Zhuang, Fairness-aware dynamic rate control and flow scheduling for network utility maximization in network service chain, IEEE J. Sel. Areas Commun. 37 (5) (2019) 1059–1071.
[12] B. Liu, X. Huang, Q. Li, Z. Huang, Y. Sun, W. Li, K. Chen, CEIO: a cache-efficient network I/O architecture for NIC-CPU data paths, in: Proceedings of the ACM SIGCOMM 2025 Conference, 2025, pp. 381–394.
[13] Y. Lin, X. Jin, X. Liu, Z. Zhang, A. Rezaeipanah, LPPIF: latency-aware placement of parallelized service function chains through instance sharing and flexible resource allocation, J. Netw. Syst. Manag. 33 (4) (2025) 98.
[14] Y. Liu, X. Shang, Y. Yang, Joint SFC deployment and resource management in heterogeneous edge for latency minimization, IEEE Trans. Parallel Distrib. Syst. 32 (8) (2021) 2131–2143.
[15] J. Zhang, P. Chen, X. Yang, H. Wu, W. Li, Distance-sensitive task allocation for mobile crowdsensing: a reinforcement learning-based mechanism, IEEe Trans. Netw. Sci. Eng. 13 (2026) 9500–9517.
[16] X. He, Z. Yang, D. Li, J. Zeng, T. Jiang, S. Chen, Dual-stage reinforcement learning-based beam tracking for integrated sensing and communications in V2I scenarios, IEEe Trans. Wirel. Commun. 25 (2026) 10759–10774.
[17] N. Qi, H. Jiang, S. Xiao, D. Wu, F. Zhou, C. Li, S. Jin, Robust and energy-efficient multi-UAV trajectory planning for data collection: a game-theoretic and deep reinforcement learning approach, IEEe Trans. Green. Commun. Netw. 10 (2026) 3086–3101.
[18] J. Mo, J. Walrand, Fair end-to-end window-based congestion control, IEEE/ACM Trans. Netw. 8 (5) (2002) 556–567.
[19] W. Dai, T. Cui, T. Zhang, B. Chen, VFL+: low-coupling vertical federated learning with privileged information paradigm, IEEe Trans. Emerg. Top. Comput. Intell. 9 (5) (2025) 3533–3547.
[20] Y. Li, S. Wang, T. Chang, Differentially private federated stochastic primal-dual learning for internet of vehicles, IEEe Internet Things J. 12 (11) (2025) 17034–17050.

[21] Y. Tian, Z. Zhu, X. Zhao, X. Chen, W. Huang, X. Zhang, A dynamic and heterogeneous representation for topology optimization using evolutionary algorithms [Research Frontier], IEEe Comput. Intell. Mag. 20 (4) (2025) 71–82.
[22] S. Shao, Y. Tian, Y. Zhang, X. Zhang, Knowledge learning-based dimensionality reduction for solving large-scale sparse multiobjective optimization problems, IEEe Trans. Cybern. 55 (7) (2025) 3471–3484.
[23] K. Zhang, Y. Wang, U.A. Bhatti, Y. Zhou, M. Jin, Enhanced ransomware attacks detection using feature selection, sensitivity analysis, and optimized hybrid model, J. Big Data 12 (1) (2025) 245.
[24] F. Ning, Z. Li, J. Lu, Y. Wang, Y. Niu, Y. Shi, 3D CAD model dynamic clustering based on inertial feature encoder, Appl. Soft Comput. 182 (2025) 113627.
[25] W. Ma, J. Wu, D. Song, Z. Chen, T. Huang, HVAE-DC: a hierarchical variational autoencoder-based deep clustering model for multi-level driving behavior, Expert Syst. Appl. 305 (2026) 130882.
[26] Y. Zhang, S. Cai, Z. Yang, S. Feng, Y. Ji, H. Zhang, One-shot federated clustering of non-independent completely distributed data, IEEe Internet Things J. 13 (2026) 14964–14978.
[27] H. Lin, C. Pan, Q. Zhang, S. Wang, Y. Zhang, S. Qiao, J. Wang, Optical-flow-guided recursive prediction-refinement neural network for particle image velocimetry, IEEE Trans. Instrum. Meas. 75 (2026) 1–16.
[28] X. Liu, C. Fischione, Coordinated beamforming for multi-cell ISAC using graph neural networks, IEEe Trans. Wirel. Commun. 25 (2026) 5876–5889.
[29] D. Huang, X. Shi, Y. Feng, C. Zhao, J. Wen, M. Shang, Automatic compiler tuning for inlining with machine learning, Intern. J. Pattern Recognit. Artif. Intell. 39 (11) (2025) 2552016.
[30] L. Qu, C. Assi, K. Shaban, M.J. Khabbaz, A reliability-aware network service chain provisioning with delay guarantees in NFV-enabled enterprise datacenter networks, IEEE Trans. Netw. Serv. Manag. 14 (3) (2017) 554–568.
[31] X. Liu, Y. Hou, H. Liu, Discrete spider monkey optimization algorithm for latency-sensitive VNF deployment and resource allocation, Int. J. Commun. Syst. 37 (2) (2024) e5640.
[32] D. Dietrich, A. Abujoda, A. Rizk, P. Papadimitriou, Multi-provider service chain embedding with nestor, IEEE Trans. Netw. Serv. Manag. 14 (1) (2017) 91–105.
[33] A. Leivadeas, G. Kesidis, M. Falkner, I. Lambadaris, A graph partitioning game theoretical approach for the VNF service chaining problem, IEEE Trans. Netw. Serv. Manag. 14 (4) (2017) 890–903.
[34] V.R. Chintapalli, R. Partani, B.R. Tamma, Energy efficient and delay aware deployment of parallelized service function chains in NFV-based networks, Comput. Netw. 243 (2024) 110289.
[35] J. Luo, J. Li, L. Jiao, J. Cai, On the effective parallelization and near-optimal deployment of service function chains, IEEE Trans. Parallel Distrib. Syst. 32 (5) (2020) 1238–1255.
[36] C. Zhou, B. Zhao, F. Tang, B. Han, B. Wang, Dynamic multi-objective service function chain placement based on deep reinforcement learning, IEEE Trans. Netw. Serv. Manag. 22 (1) (2024) 15–29.
[37] K. Qu, W. Zhuang, X. Shen, X. Li, J. Rao, Dynamic resource scaling for VNF over nonstationary traffic: a learning approach, IEEe Trans. Cogn. Commun. Netw. 7 (2) (2020) 648–662.
[38] X. Yu, R. Wang, J. Hao, Q. Wu, C. Yi, P. Wang, D. Niyato, Priority-aware deployment of autoscaling service function chains based on deep reinforcement learning, IEEe Trans. Cogn. Commun. Netw. 10 (3) (2024) 1050–1062.
[39] T. Wang, Q. Fan, X. Li, X. Zhang, Q. Xiong, S. Fu, M. Gao, Drl-sfcp: adaptive service function chains placement with deep reinforcement learning, in: Proceedings of the ICC2021-IEEE International Conference on Communications, IEEE, 2021, pp. 1–6.
[40] Y. Zhang, F. Zhang, S. Tong, A. Rezaeipanah, A dynamic planning model for deploying service functions chain in fog-cloud computing, J. King Saud Univ.-Comput. Inf. Sci. 34 (10) (2022) 7948–7960.
[41] H. Huang, J. Tian, G. Min, H. Yin, C. Zeng, Y. Zhao, D.O. Wu, Parallel placement of virtualized network functions via federated deep reinforcement learning, IEEE/ACM Trans. Netw. 32 (4) (2024) 2936–2949.
[42] C. Guo, A. Rezaeipanah, Dynamic service function chains placement based on parallelized requests in edge computing environment, Trans. Emerg. Telecommun. Technol. 35 (1) (2024) e4905.
[43] K. Zhang, Y. Zhou, S. Zhang, H. Wang, A. Qourbani, Towards deploying SFC with parallelized VNFs under resource demand uncertainty in mobile edge computing, J. King Saud Univ.-Comput. Inf. Sci. 35 (7) (2023) 101619.
[44] Z. Sharif, M.B. Jasser, K.L.A. Yau, A. Amphawan, Advancements and challenges in latency-optimized joint SFC placement and routing: a comprehensive review and future perspectives, Int. J. Syst. Assur. Eng. Manag. 16 (3) (2025) 1072–1105.
[45] G.L. Santos, D.D.F. Bezerra, E.D.S. Rocha, L. Ferreira, A.L.C. Moreira, G. E. Gonçalves, P.T. Endo, Service function chain placement in distributed scenarios: a systematic review, J. Netw. Syst. Manag. 30 (1) (2022) 4.
[46] D. Li, P. Hong, K. Xue, Virtual network function placement considering resource optimization and SFC requests in cloud datacenter, IEEE Trans. Parallel Distrib. Syst. 29 (7) (2018) 1664–1677.
[47] D. Zhang, L. Wang, A. Rezaeipanah, Towards deploying parallelized service function chains under dynamic resource request in multi-access edge computing, IEEE Trans. Netw. Serv. Manag. (2024), https://doi.org/10.1109/TNSM.2024.3520911.
[48] F. Bari, S.R. Chowdhury, R. Ahmed, R. Boutaba, O.C.M.B. Duarte, Orchestrating virtualized network functions, IEEE Trans. Netw. Serv. Manag. 13 (4) (2016) 725–739.
[49] B. Li, B. Cheng, X. Liu, M. Wang, Y. Yue, J. Chen, Joint resource optimization and delay-aware virtual network function migration in data center networks, IEEE Trans. Netw. Serv. Manag. 18 (3) (2021) 2960–2974.
[50] J. Li, W. Shi, H. Wu, S. Zhang, X. Shen, Cost-aware dynamic SFC mapping and scheduling in SDN/NFV-enabled space–air–ground-integrated networks for Internet of Vehicles, IEEe Internet Things J. 9 (8) (2021) 5824–5838.
[51] Z. Fang, J. Cao, C. Yuan, L. Tian, State and computing decoupled service function chaining for scalable cloud RANs, in: Proceedings of the ICC 2024-IEEE International Conference on Communications, IEEE, 2024, pp. 5269–5274.
[52] M. Asgarian, K. Jamshidi, A. Bohlooli, An efficient approximation algorithm for service function chaining placement in edge–cloud computing industrial internet of things, IEEe Internet Things J. 11 (7) (2024) 12815–12822.
[53] Y. Liu, Y. Lu, X. Li, Z. Li, W. Qiao, Y. Zhang, D. Zhao, A Lagrangian-relaxation-based approach for service function chain dynamic orchestration for the Internet of Things, IEEe Internet Things J. 8 (23) (2021) 17071–17089.
[54] N.H. Thanh, N.T. Kien, N. Van Hoa, T.T. Huong, F. Wamser, T. Hossfeld, Energy-aware service function chain embedding in edge–cloud environments for IoT applications, IEEe Internet Things J. 8 (17) (2021) 13465–13486.
[55] P. Chen, Y. Song, Y. Xia, Adaptively diagnosing system faults in microservice architecture: an autonomous predictive model construction framework, Future Gener. Comput. Syst. (2025) 108256.
[56] X. Cheng, Y. Chen, X. Yang, H. Zhou, L. Luo, D. Guo, Memory-efficient programmable packet parsing for multi-tenant terabit networks, Comput. Netw. 264 (2025) 111240.
[57] Y. Yao, J. Sun, P. Miao, G. Chen, R. Tafazolli, Energy-efficient beamforming for STAR-RIS-aided ISAC with hardware impairments: a generative AI-Enabled DRL method, IEEe Trans. Wirel. Commun. 25 (2026) 17607–17619.
[58] Z. Xiao, Y. Chen, T. Li, S. Tian, G. Min, Z. Tang, K. Li, Toward secure task offloading in vehicular edge computing networks, IEEe Trans. Dependable Secure Comput. (2026), https://doi.org/10.1109/TDSC.2026.3697937.
[59] L. Jin, Y. Zhang, Y. Li, S. Wang, H.H. Yang, J. Wu, M. Zhang, MoE2: optimizing collaborative inference for edge large language models, IEEE Trans. Netw. 34 (2026) 4637–4651.
[60] F. Wang, H. Yang, S. Zhang, X. He, L. Zhu, T.Q.S. Quek, GEMS: GAT-embedded mobility for dual connectivity in integrated terrestrial and non-terrestrial networks, IEEe Trans. Mob. Comput. (2026), https://doi.org/10.1109/TMC.2026.3681814.
[61] F. Zahedi, M. Mollahoseini Ardakani, A. Heidary-Sharifabad, DSPVR: dynamic SFC placement with VNF reuse in fog-cloud computing using deep reinforcement learning, J. Ambient. Intell. Humaniz. Comput. 14 (4) (2023) 3981–3994.
[62] M.B. Taha, Y. Sanjalawe, A. Al-Daraiseh, S. Fraihat, S.R. Al-E'mari, Proactive auto-scaling for service function chains in cloud computing based on deep learning, IEEe Access 12 (2024) 38575–38593.
[63] Y. Liu, J. Zhang, Service function chain embedding meets machine learning: deep reinforcement learning approach, IEEE Trans. Netw. Serv. Manag. 21 (3) (2024) 3465–3481.
[64] S. Xu, Y. Li, S. Guo, C. Lei, D. Liu, X. Qiu, Cloud–edge collaborative SFC mapping for industrial IoT using deep reinforcement learning, IEEe Trans. Industr. Inform. 18 (6) (2021) 4158–4168.
[65] M.R. Abedi, M.E. Fasanghari, M. Akbari, N. Mokari, H. Yanikomeroglu, Dynamic pricing in multi-tenant MANO with resource sharing: a Stackelberg game approach, IEEE Open J. Commun. Soc. 5 (2024) 7002–7021.
[66] M. Pourghasemian, M.R. Abedi, S.S. Hosseini, N. Mokari, M.R. Javan, E. A. Jorswieck, AI-based mobility-aware energy efficient resource allocation and trajectory design for NFV enabled aerial networks, IEEe Trans. Green. Commun. Netw. 7 (1) (2022) 281–297.
[67] M. Akbari, M.R. Abedi, R. Joda, M. Pourghasemian, N. Mokari, M. Erol-Kantarci, Age of information aware VNF scheduling in industrial IoT using deep reinforcement learning, IEEE J. Sel. Areas Commun. 39 (8) (2021) 2487–2500.
[68] Y.C. Wang, S.H. Wu, Efficient deployment of virtual network functions to achieve load balance in cloud networks, in: Proceedings of the 2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS), IEEE, 2022, pp. 1–6.